

Design of a Low-Power Asynchronous SAR ADC in 45 nm CMOS Technology

Muhammad Aldacher
San Jose State University

Advisor: Dr. Sotoudeh Hamedi-Hagh

Co-advisor: Dr. Sang-Soo Lee

Design of a Low-Power Asynchronous SAR ADC in 45 nm CMOS Technology

Muhammad Aldacher
San Jose State University

Advisor: Dr. Sotoudeh Hamedi-Hagh

Co-advisor: Dr. Sang-Soo Lee

TABLE OF CONTENTS

1. ABSTRACT	2
2. INTRODUCTION	2
2.1 MOTIVATION	2
2.2 BASIC SAR CONCEPT	3
2.3 DIFFERENTIAL SAR ARCHITECTURE.....	6
2.4 ASYNCHRONOUS SAR OPERATION.....	7
3. DESIGN.....	8
3.1 INTERNAL CLOCK GENERATOR.....	8
3.2 BOOTSTRAPPED SAMPLE-AND-HOLD CIRCUIT	11
3.3 CAPACITIVE DAC.....	13
3.4 DYNAMIC COMPARATOR	15
3.5 SAR LOGIC.....	17
3.6 OUTPUT STAGE	19
3.6.1 Output Register	20
3.6.2 Output DAC	21
4. RESULTS AND DISCUSSION	21
4.1 IDEAL MODELS.....	22
4.2 INDIVIDUAL BLOCKS.....	24
4.2.1 Bootstrapped Switch	25
4.2.2 Capacitive DAC	27
4.2.3 Comparator.....	29
4.3 WHOLE ASYNCHRONOUS SAR ADC SYSTEM	31
5. CONCLUSION	36
APPENDIX A: VERILOG-A CODES AND IDEAL MODELS	37
A.1: SYNCHRONOUS SAR ADC:.....	37
A.1.1 Sample-And-Hold:	37
A.1.2 Clocked Comparator:	37
A.1.3 SAR Logic:.....	38
A.1.4 Output Register (8-bit):	40
A.1.5 DAC (8-bit):	40
A.2: ASYNCHRONOUS SAR ADC:.....	41
A.2.1 Internal Clock Generator:	41
A.2.2 SAR Logic:.....	42
A.2.3 D-Flipflop:.....	42
A.2.4 Inverter Gate:	43
A.2.5 XOR Gate:	43
A.2.6 AND Gate:	44
A.2.7 Multiplexer (2-to-1):.....	44
A.2.8 Delay Cell (1ns):	45
REFERENCES	46

1. Abstract

This project proposes a 1V 8-bit asynchronous successive approximation register (SAR) analog-to-digital converter (ADC) implemented in 45nm CMOS technology. The asynchronous SAR ADC system consists of an internal-clock generator, a bootstrapped sample-and-hold switch, a capacitive digital-to-analog converter (DAC), a dynamic comparator, and a SAR logic. Taking a 64-point FFT on the output of the SAR ADC and with an input-signal of 1.2V differentially, the maximum ENOB achieved at 20 MHz sampling frequency is 7.35 bits, with an SNR of 46 dB & a total power consumption of 49.124 uW. The figure-of-merit achieved, based on Walden's calculation, is 15 fJ/conv. This SAR ADC system can be used in systems that mainly require low power with medium resolution and medium speed like in computing-in-memory cores for AI applications and in sensors for biomedical applications.

2. Introduction

2.1 Motivation

The SAR ADC architecture has become an attractive solution in recent low-power applications since it offers a good compromise between medium resolution and speed and low power consumption [1]. Typical SAR ADCs utilize the binary search algorithm in a feedback system to produce a sample of data after a series of sequential iterations, resulting in a relatively lower sample rate compared to pipeline and flash ADCs, as shown in Fig.1. But since SAR ADCs don't require power-hungry analog circuits like Opamps, neither do they use a large number of comparators per ADC, the total power of the building blocks of the SAR ADCs can scale down well with technology. The power efficiency of SAR ADCs is useful for systems that require the use of a large number of ADCs and operate at speeds around a few KHz or a few MHz, like in computing-in-memory cores (CIMs) used in artificial intelligence systems [2] and biomedical signal processing units [3].

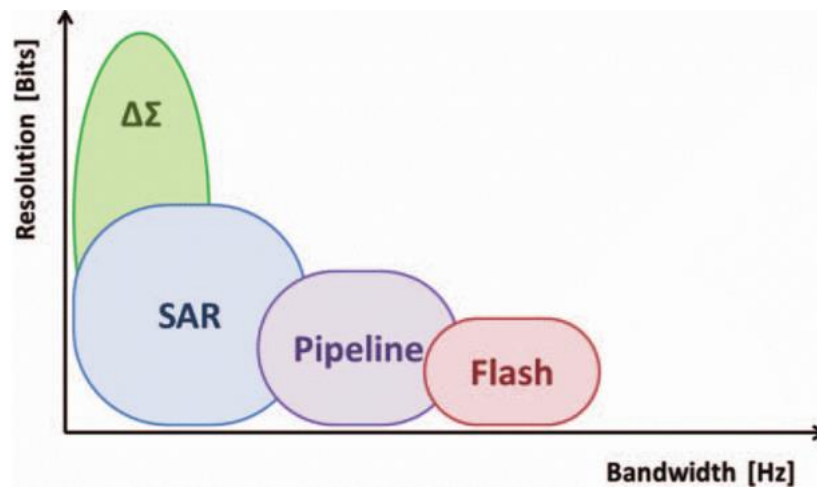


Fig.1. Popular ADC Architectures' Resolutions VS Bandwidth

In terms of control schemes, there are two types of SAR ADC architectures; the synchronous SAR and the asynchronous SAR. Unlike the synchronous architecture, the process of the asynchronous SAR ADC is controlled by internally generated clock signals, providing the same sample rate as the typical synchronous SAR ADC while using a slower external sampling clock signal. In synchronous SAR ADCs, at least N clock cycles are needed per conversion, while in asynchronous SAR ADCs, the whole conversion process occurs within 1 cycle of the external clock. The idea is to use internal conditions to cause the sequential binary search process to start and finish within 1 clock cycle. By removing the need for an oversampled clock signal, the dynamic power and the complexity of the logic circuits can be further reduced [4]. Another advantage for the asynchronous architecture is that it is possible to further decrease the conversion time at a given resolution, since each comparison depends on the internal conditions of the circuits in the system, rather than the period of the sampling clock [5].

2.2 Basic SAR Concept

In a typical N -bit SAR ADC, the conversion operation consists of 3 main processes: sample-and-hold, comparison for N times, and then output the digital code. To explain the basic operation, let's see the example of a single-ended capacitive DAC of a 3-bit SAR ADC, shown in Fig.2. The common-mode voltage V_{CM} is assumed to be zero for easier calculations.

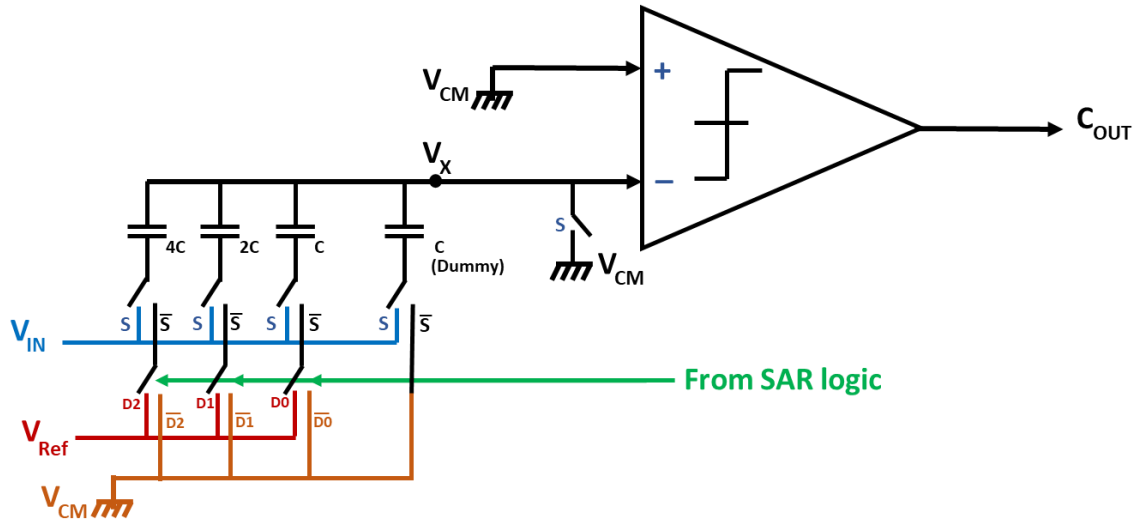


Fig.2. Capacitive DAC for a 3-bit SAR ADC

At the beginning of any conversion operation, the input signal V_{IN} is sampled to the bottom plates of the capacitors using the S switches, while the upper plate (node V_X) is connected to V_{CM} . The total charge stored on the capacitors is equal to V_{IN} multiplied by the sum of the capacitors. The voltage and the charge at the node V_X are shown in equations 1 and 2.

During sampling phase: $V_X = V_{CM} = 0$ (1)

$$Q_X = -V_{IN} \cdot 8C \quad (2)$$

After the sample phase, the top plate of the capacitors is disconnected from V_{CM} , while the bottom plate is disconnected from V_{IN} and is changed according to the digital code coming from the SAR logic. In the first comparison, the MSB is set to 1 making the DAC code 100 initially. Through charge distribution and voltage division, V_X changes as shown in equation 4 and compared to V_{CM} (0 in this example). This is equivalent to V_{IN} being compared to $\frac{1}{2} V_{REF}$. If $V_{IN} > \frac{1}{2} V_{REF}$ ($V_X < 0$), the comparator outputs a 1, otherwise, the comparator gives a 0.

Comparison 1 (DAC=100): $\Delta V = \frac{4C}{4C + 2C + C + C}$ (3)

$$V_X = -V_{IN} + \Delta V = -V_{IN} + \frac{1}{2} V_{REF} \quad (4)$$

In the next comparison, the MSB will take the value of the comparator's output D_2 and the MSB-1 bit is set to 1. V_{IN} now is compared in either the upper half of V_{REF} if $D_2 = 1$ or the lower half of V_{REF} if $D_2 = 0$. After this comparison, the MSB-1 bit takes the value of the comparator D_1 and MSB-2 bit is set to 1. After $N=3$ comparisons, the SAR ADC outputs $D_2D_1D_0$ as the final binary code that translates to the voltage level V_{DAC} given in equation 9. Fig.3. shows the change in the node voltages of the capacitors in each stage of the conversion process. The binary search process is shown in Fig.4. The binary tree in Fig.5. shows the all the possible outcomes of this 3-bit SAR system. The stages of this binary tree can be extended to N stages for an N -bit SAR ADC.

Comparison 2 (DAC= D_210): $V_{DAC} = \frac{(D_2 \cdot 4C) + 2C}{4C + 2C + C + C}$ (5)

$$V_X = -V_{IN} + V_{DAC} = -V_{IN} + \frac{D_2}{2} V_{REF} + \frac{1}{4} V_{REF} \quad (6)$$

Comparison 3 (DAC= D_2D_11): $V_{DAC} = \frac{(D_2 \cdot 4C) + (D_1 \cdot 2C) + 1C}{4C + 2C + C + C}$ (7)

$$V_X = -V_{IN} + V_{DAC} = -V_{IN} + \frac{D_2}{2} V_{REF} + \frac{D_1}{4} V_{REF} + \frac{1}{8} V_{REF} \quad (8)$$

Final output (DAC= $D_2D_1D_0$) is equivalent to:

$$V_{DAC} = \frac{D_2}{2} V_{REF} + \frac{D_1}{4} V_{REF} + \frac{D_0}{8} V_{REF} \quad (9)$$

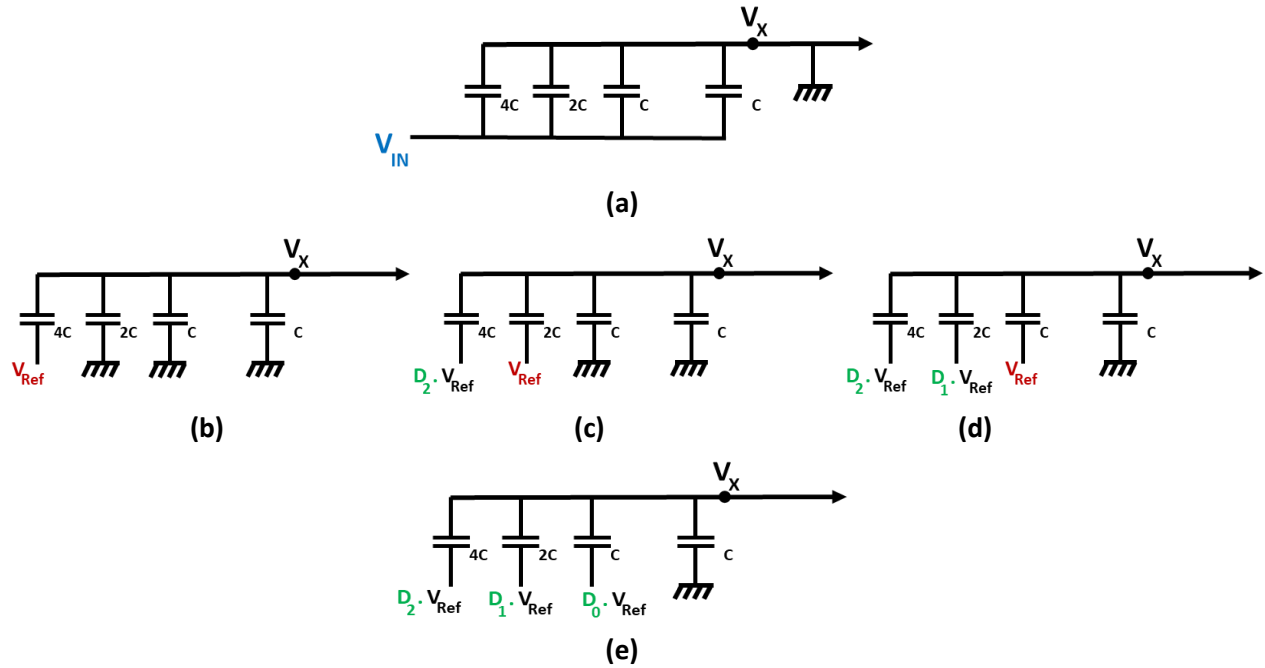


Fig.3. Node voltages of the DAC capacitors for each stage
(a) Sampling phase, (b) Comparison 1, (c) Comparison 2, (d) Comparison 3, (e) Final binary output

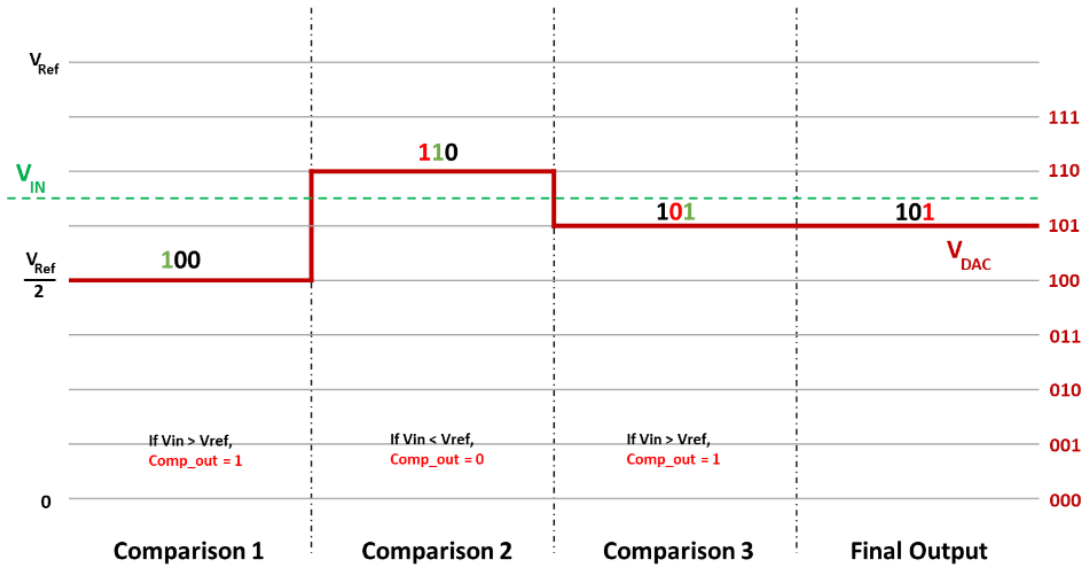


Fig.4. Binary search process in the 3-bit SAR system

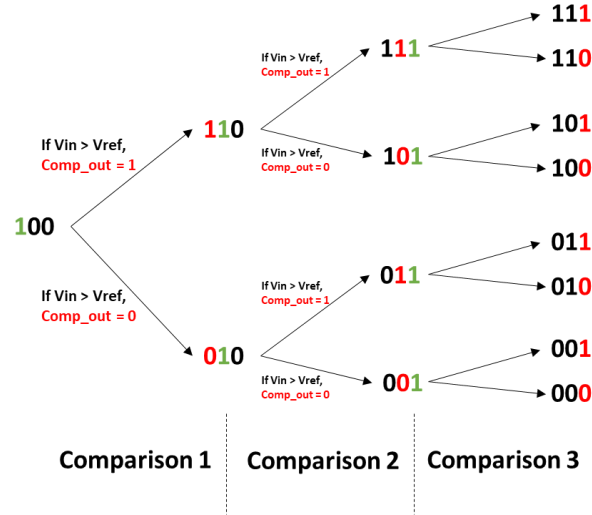


Fig.5. Binary tree showing all the possible outputs of a 3-bit SAR ADC

2.3 Differential SAR Architecture

The idea presented in the previous section can be extended to a differential configuration. Fig.6. shows an 8-bit differential capacitive DAC. In the differential DAC, negative and the positive references are used as defined in equations 10 and 11, respectively. The 2 input nodes to the comparator are compared in each iteration, so if $(V_{XP} - V_{XN}) > 0$, then the comparator will output a 1, otherwise, it produces a 0. The differential configuration has many advantages over the single-ended configuration like better common-mode rejection ratio (CMRR), the reduction of even harmonic distortion, and the doubling of input voltage range [6], but at the expense of larger layout area since most of the components are doubled for the differential configuration.

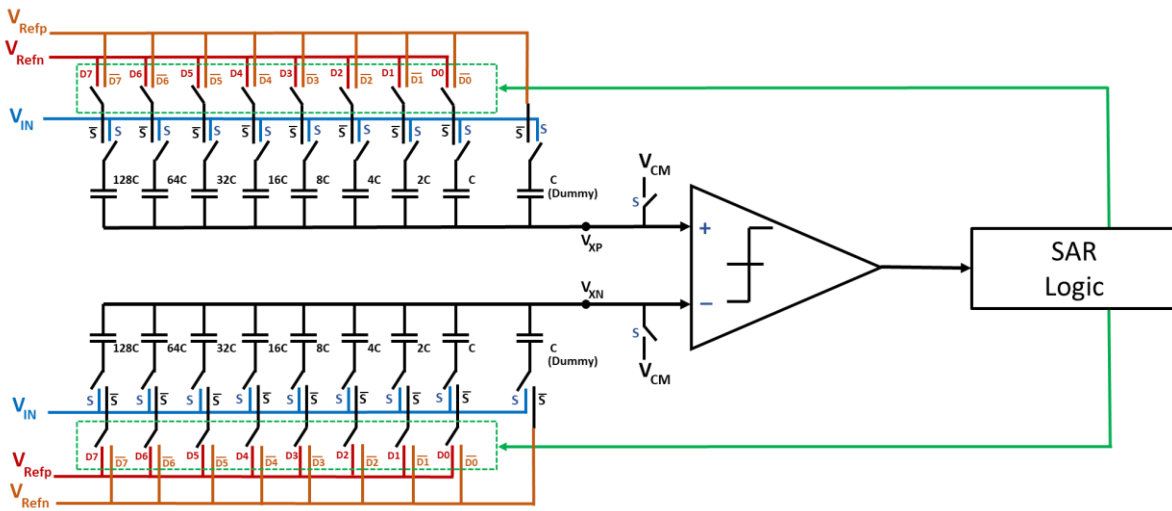


Fig.6. Differential capacitive DAC of an 8-bit SAR ADC

$$V_{Refn} = V_{CM} - \frac{V_{Ref}}{2} \quad (10)$$

$$V_{Refp} = V_{CM} + \frac{V_{Ref}}{2} \quad (11)$$

2.4 Asynchronous SAR Operation

The asynchronous SAR ADC is called “asynchronous” because the blocks in this SAR system don’t operate with the same uniform clock signal. As shown in Fig. 7, each block produces a signal that signals another block to change state. The external clock only triggers the internal-clock generator to begin the conversion process. The generator produces 2 internal signals, “Clk_Sample”, which is a pulse used for the sampling phase by the sample-and-hold (S/H) block, and “Clk_SAR”, which controls the comparator and the SAR code that changes the DAC voltage. The conversion process starts with the S/H block sampling the input, then “Clk_SAR” goes high which prompts the SAR logic to apply the DAC code. After a certain delay (must be enough for the DAC to settle), “Clk_SAR” goes low to cause the comparator to compare and generate the output. When the comparator finishes the comparison and its output is stable, a “Ready” pulse is sent to the internal-clock generator to cause the “Clk_SAR” to toggle and repeat the cycle for the next iteration. After N cycles of “Clk_SAR”, an end-of-code (EOC) pulse is sent to an output register to send out the final DAC code of the conversion. The timing diagram of the asynchronous SAR system and the essential waveforms are shown in Fig. 8.

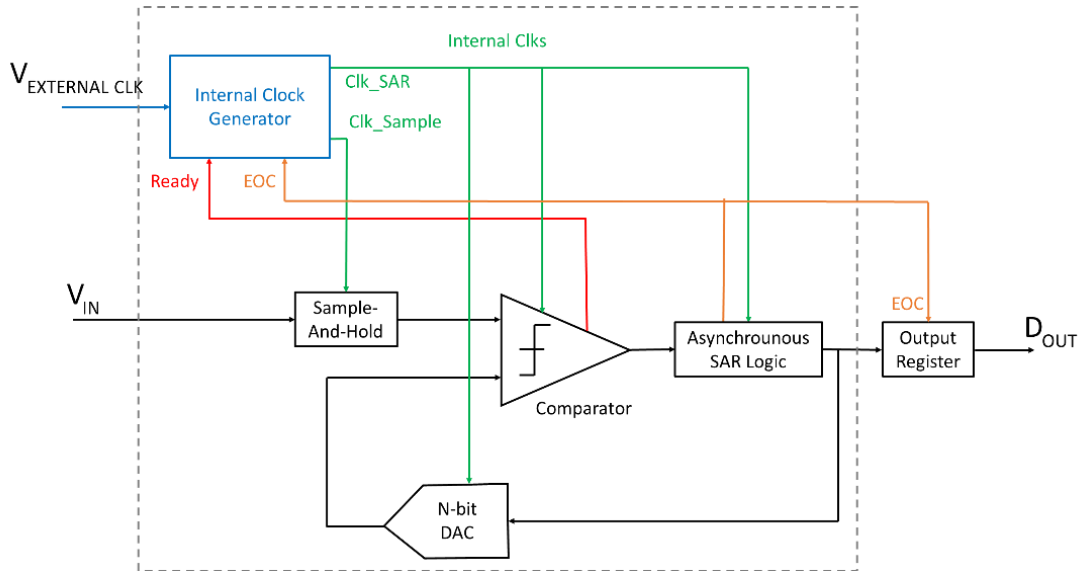


Fig.7. High-level block diagram of an asynchronous SAR ADC

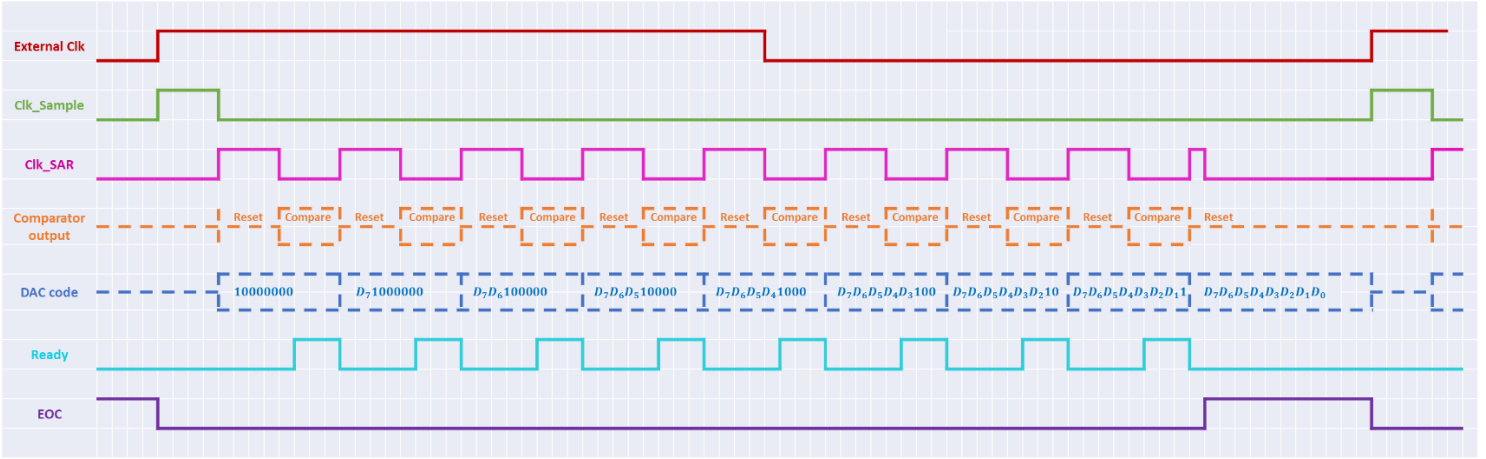


Fig.8. Timing diagram of an asynchronous SAR ADC system

3. Design

3.1 Internal Clock Generator

The internal clock generator is the block responsible for providing the rest of the system with the clock signals to operate. This block produces 2 output signals, “clk_sample” for the sample phase and “clk_sar” for the comparison phases, as explained before. There are 3 inputs to this block, “clk_ext” which is the system’s external clock, “Ready” which is produced by the comparator once the comparison is done to tell the system that it is ready for the next iteration, and “EOC” which is produced by the SAR logic block once all the N bits are generated. The rising edge of the external clock is only used to prompt the beginning of the conversion cycle, then the rest of the inputs and the outputs control the rest of the process till the “EOC” is generated, concluding the conversion.

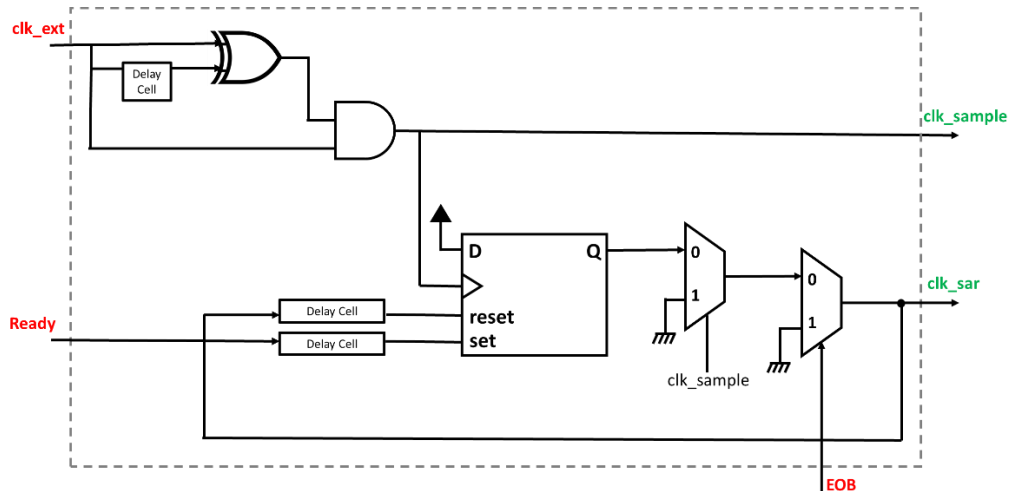


Fig.9. block diagram of the internal clock generator

The circuit implementation of the internal clock generator is shown in Fig. 10. Part A is the part that creates the “clk_sample” signal. The delay chosen in this section is enough for the S/H circuit to settle when charging the DAC capacitors during the sampling phase. The XOR gate creates a pulse at each transition of the external clock and the AND gate only passes the pulse when “clk_ext” is high, as shown in Fig. 11. Part B is responsible for toggling “clk_SAR” after the sampling phase is done. The 2-to-1 multiplexers are used to prevent “clk_SAR” from toggling and to stay 0 during the sampling phase (clk_sample = 1) or after the conversion is done (EOC = 1). The D input of the flipflop is kept high, so that the Q output would initially be high, consequently causing “clk_SAR” to go high once the sampling phase ends (clk_sample becomes 0). Part C controls the toggling of “clk_SAR” through the set and reset inputs of the flipflop, while ensuring that set and reset are never high at the same time (invalid state). The “clk_SAR” going high triggers the reset signal, which pulls “clk_SAR” low after a certain delay, and the “Ready” going high triggers the set signal, which pulls “clk_SAR” high after a certain delay. The flipflop used is a true-single phase clock (TSPC) flipflop, which is discussed in section 3.5, and the rest of the digital gates are CMOS gates, as shown in Fig. 12. The implementation of the delay cell is shown in Fig. 13.

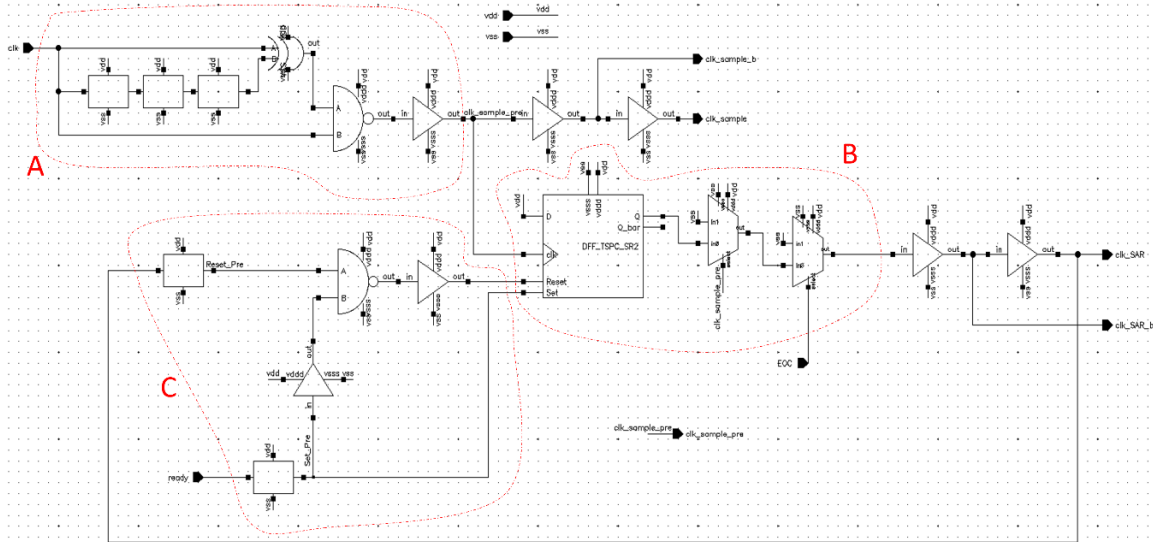


Fig.10. Schematics of the circuit implementation of the internal clock generator

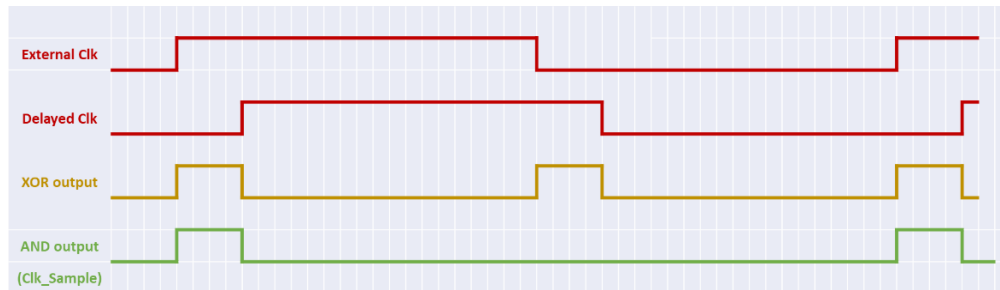
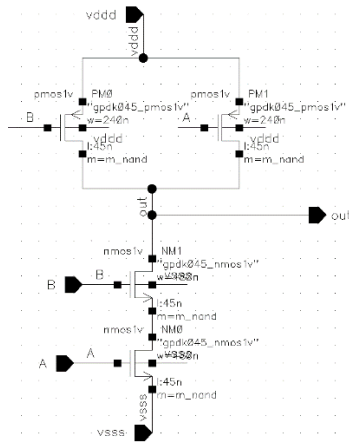
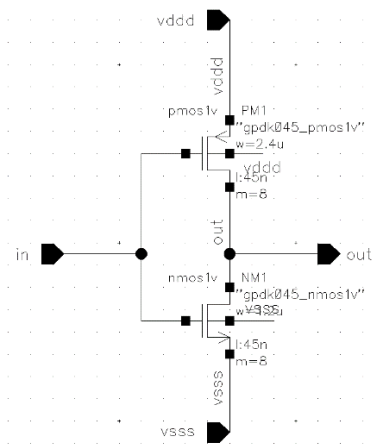


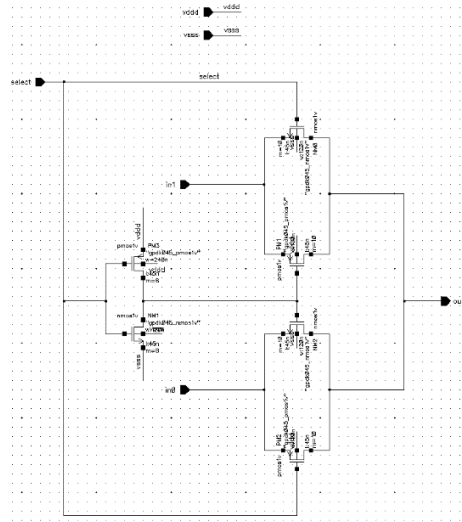
Fig.11. Timing diagram for “clk_sample” generation circuit



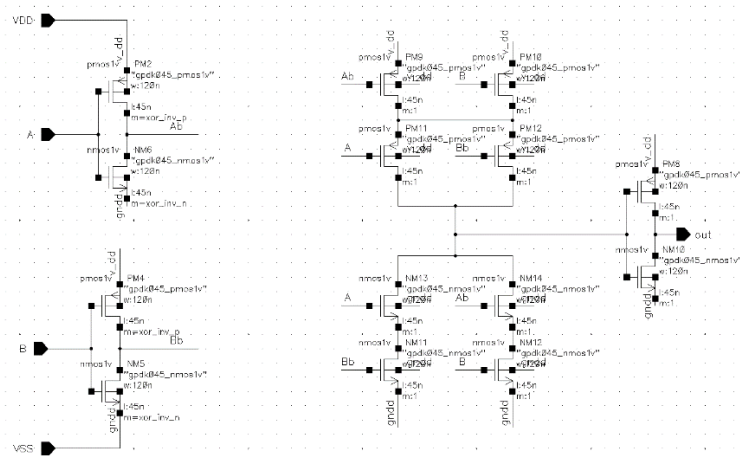
(a)



(b)



(c)



(d)

Fig.12. Digital gates used in the internal clock generator block
(a) NAND, (b) Inverter, (c) 2-to-1 Mux, (d) XOR

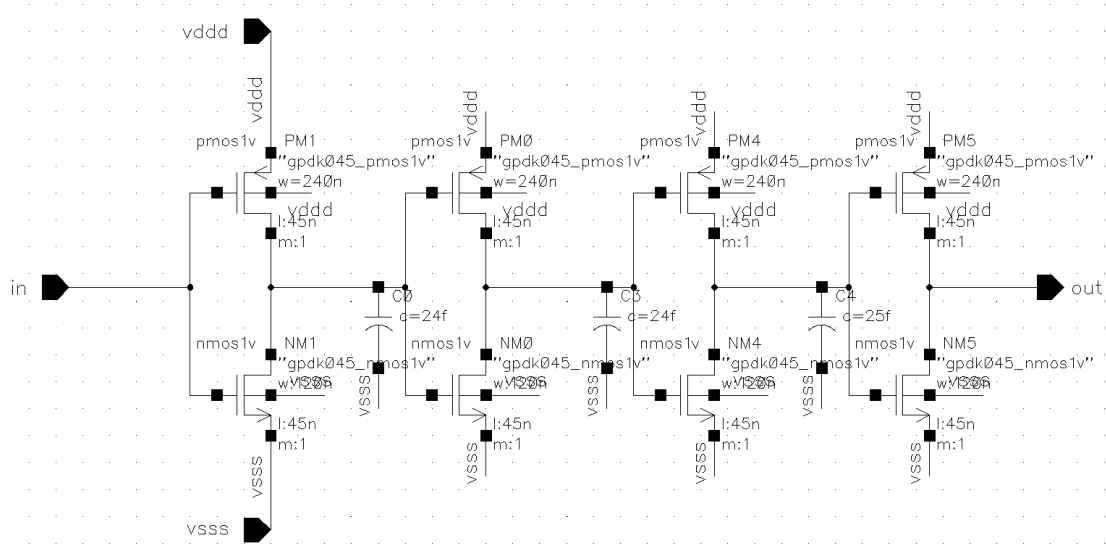


Fig.13. Delay cell circuit implementation

3.2 Bootstrapped Sample-And-Hold Circuit

The sample-and-hold circuit is basically a switch that captures the value of an analog signal at a certain moment and holds it constant for a certain amount of time. In most circuits, the switch is implemented by an NMOS pass gate, a PMOS pass gate, or a CMOS transmission gate, shown in Fig. 14. But these switches have non-idealities that affect the quality of the sampled signal. MOSFET switches are in triode state when they are ON resulting in a non-zero on-resistance R_{ON} that depends on the input signal, as shown in equation 12. Another effect that depends on the input is the injection of the charge in the inversion layer into the load capacitor when the switch is turning off. This charge Q_{CH} is given in equation 13. Additionally, there is also clock feedthrough that arises from the presence of parasitic capacitances from the gate of the switch to the output node. The switching activity of the clock translates to a voltage disturbance given by equation 14. Fig. 15 illustrates the effect of these non-idealities in MOSFET switches [7].

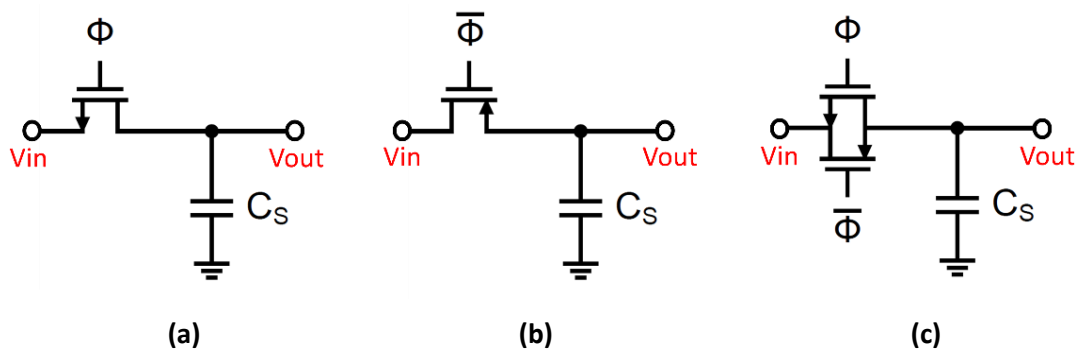


Fig.14. Basic sampling circuits (a) NMOS switch, (b) PMOS switch, (c) CMOS Transmission gate

$$R_{ON} = \frac{1}{\mu_n \cdot C_{OX} \cdot W/L \cdot (V_{GS} - V_{TH})} = \frac{1}{\mu_n \cdot C_{OX} \cdot W/L \cdot ((V_{DD} - V_{IN}) - V_{TH})} \quad (12)$$

$$Q_{CH} = W \cdot L \cdot C_{OX} \cdot (V_{GS} - V_{TH}) = W \cdot L \cdot C_{OX} \cdot ((V_{DD} - V_{IN}) - V_{TH}) \quad (13)$$

$$\Delta V_{OUT} = V_{CLK} \cdot \frac{W \cdot C_{OV}}{W \cdot C_{OV} + C_S} \quad (14)$$

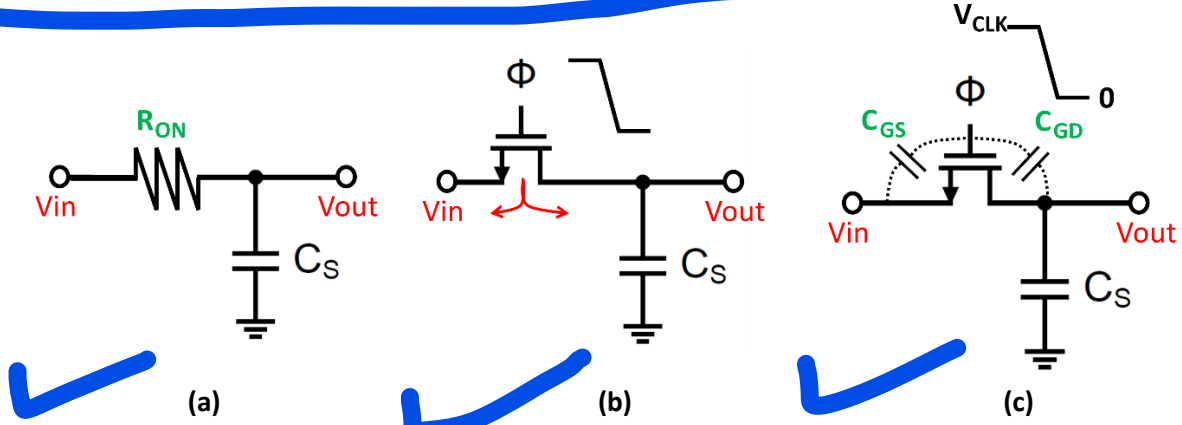


Fig.15. Non-idealities in MOSFET switches
(a) input-dependent R_{ON} , (b) Charge injection, (c) Clock feedthrough

The bootstrapped switch help mitigate some of these non-idealities by keeping the V_{GS} of the switch fixed during the sampling phase. In the basic illustration in Fig. 16 (a), Φ_2 switches are ON during the hold phase, turning the main switch MNSW completely off by connecting its gate to ground and charging the capacitor C_{OFFSET} to a voltage of $V_C = V_{DD} - V_{SS}$ [8]. During the sampling phase, Φ_1 switches are ON while Φ_2 switches are OFF, causing the capacitor to act as a constant battery fixing V_{GS} to be equal to V_C . Fig. 16 (b) shows how the gate voltage V_g and the source voltage V_s of MNSW are bootstrapped allowing them to change together. Fixing V_{GS} suppresses the distortion due to dependence of R_{ON} and Q_{CH} on the input signal. Also, the width W of MNSW does not need to be large to further reduce R_{ON} 's effect, which in turn helps reducing clock feedthrough as well since the parasitic caps are smaller with smaller W .

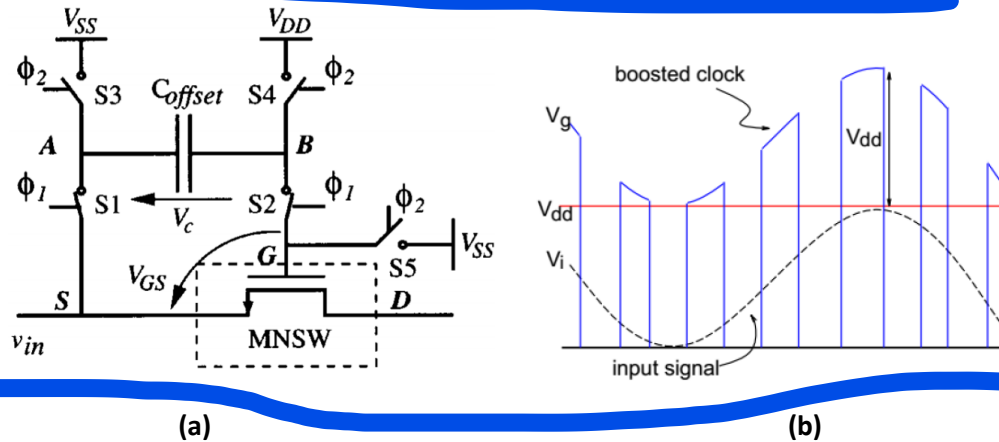


Fig.16. (a) Basic bootstrapped switch circuit, (b) V_g waveform while bootstrapped to $V_s (= V_i)$

The proposed bootstrapped circuit is shown in Fig. 17. The MOSFET devices Mn1, Mp1, Mn2, and Mp2 act as switches that are used to charge the offset capacitor during the hold phase ($\text{clk} = 0$) and to fix the V_{GS} of the main switch MnSW during the sample/track phase ($\text{clk} = 1$). The large V_{GS} ensures that MnSW is completely ON and much greater than the threshold voltage V_{TH} while the source voltage of MnSW is changing. The gate nodes of MnSW and Mp2 are tied because during sampling, V_g and Mp2's source are usually greater than V_{DD} . For Mp2 to be completely off, its gate must be tied to the highest voltage in the circuit, hence tied to V_g . The bulks of the PMOS switches Mp1 and Mp2 are connected to the highest voltage of the circuit which is the positive node of C_{OFFSET} . During the hold phase, Mn4 connects V_g to the lowest voltage in the system to ensure complete shut-off of MnSW. Mn3 is added to prevent Mn4 from having a very high V_{GD} when Mn4 is OFF. Since $V_g \geq V_{DD}$ and the gate voltage of Mn3 is V_{DD} (less than V_g), then Mn3 won't act as a strong pass device and the drain of Mn4 will be limited to a maximum of $V_{DD} - V_{TH}$. Regarding the value of C_{OFFSET} , the bigger it is, the better it can maintain a constant V_c during the sampling phase, but at the expense of larger area. With MpSW added in parallel to MnSW, the clock feedthrough effect is cancelled-out, since they both operate with 2 opposing clocks (clk & clk_b).

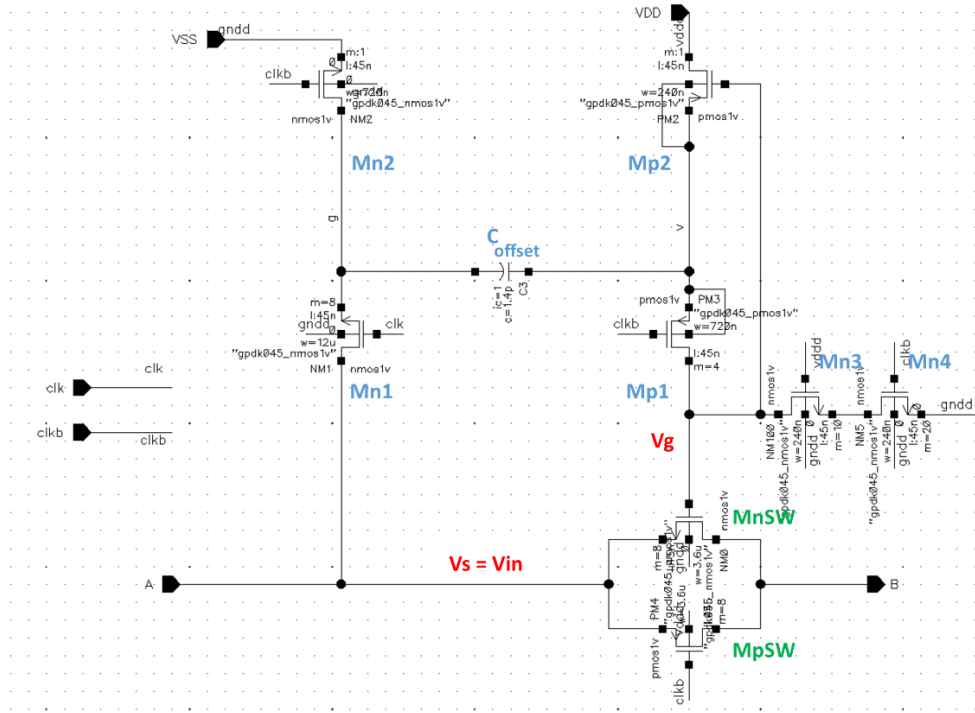


Fig.17. Schematics of the implemented bootstrapped switch circuit

3.3 Capacitive DAC

The DAC is responsible for converting the digital code coming from the SAR logic into the corresponding analog voltage level to be compared by the comparator. The conventional capacitive DAC configuration uses the charge redistribution concept in an array of N binary-weighted capacitors in addition

to a dummy capacitor. The differential capacitive DAC employs 2 of such array, as shown in Fig. 18 [9]. Other than the switches used to sample the inputs, the switches used in the DAC are CMOS transmission gates. To switch between the sampled input and then to the DAC code selection, a switch network is used as shown in Fig. 19. To make the transmission gates less resistive (smaller R_{ON}), the widths of the NMOS and PMOS transistors are increased, but this is at the expense of larger parasitic capacitances. For the DAC capacitors, increasing the unit capacitor's value decreases the thermal noise effect but also decreases the speed of the system. In the proposed DAC, an additional common-mode switch is added between V_{XP} and V_{XN} for a better and faster connection to each other and to V_{CM} during the sampling phase.

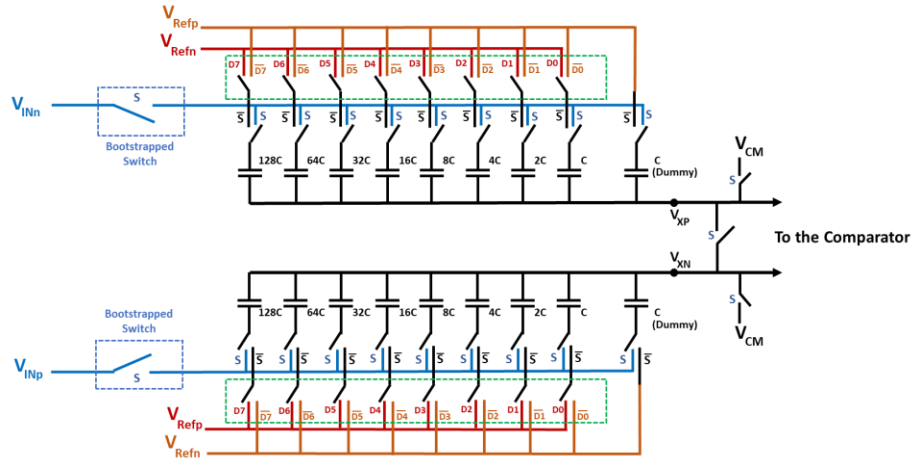


Fig.18. Proposed differential capacitive DAC

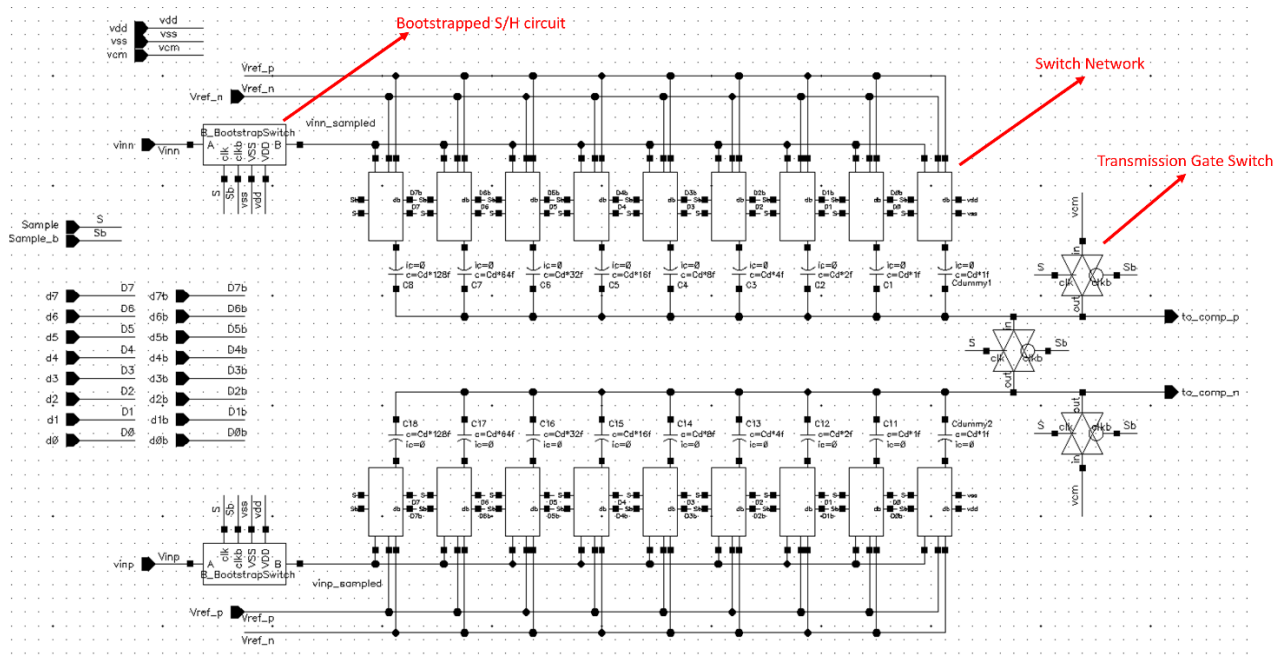


Fig.19. Schematics of the capacitive DAC circuit implementation

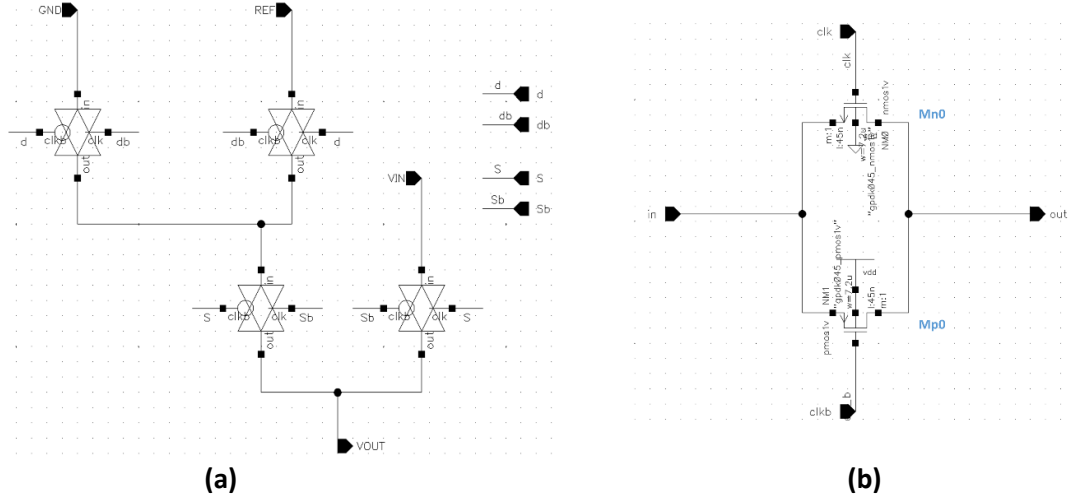


Fig.20. Schematics of the switches used in the DAC (a) Switch network, (b) Transmission gate

3.4 Dynamic Comparator

The comparator's function is to compare the analog input levels coming from the DAC and to produce a digital output bit (either high or low) representing the result of the comparison. Dynamic comparators are commonly used in ADCs due to their high speed and low power compared to static comparators or opamp-based comparators. Being a critical part of the ADC system, it is crucial to improve the key parameters of the dynamic comparator that include the gain, resolution, input-referred offset, and kickback noise.

Fig. 21. shows 2 main types of dynamic comparators, the strong-arm comparator and the two-stage comparator. The strong-arm configuration is a common circuit used in various systems like as a sense amplifier in SRAM memory blocks, a sensitive latch in digital PLLs, and a high-speed comparator in ADCs. Strong-Arm comparators are characterized by their low power consumption (as it consumes almost zero-DC current), high speed (due to its regenerative latch), and full rail-to-rail output swing [10]. On the other hand, the two-stage comparator consists of a preamp stage and a regenerative latch. Introducing the preamp stage into the comparator system separates the output stage from the input nodes reducing the input-referred offset and the kickback noise, in addition to giving an extra gain stage [11].

The proposed comparator, shown in Fig. 22, consists of a strong-arm latch (Fig. 23) [12], an RS latch (Fig. 24), and an XOR gate. During the reset phase of the strong-arm latch ($\text{clk} = 0$), Mn_tail is OFF which shuts off the tail current of the latch, and Mp3 and Mp4 are ON which causes the output nodes to be pulled high to V_{DD} . In the regeneration phase ($\text{clk} = 1$), Mn_tail is ON and the back-to-back inverters consisting of Mn1 , Mn2 , Mp1 , and Mp2 are activated. The transistors Mn3 and Mn4 control the currents in each branch according to in^+ and in^- node voltages. Assuming $\text{in}^+ > \text{in}^-$, Mn3 will pull more current in its branch, pulling out $^-$ node towards ground. Since out $^-$ is the input to the other inverter, out $^+$ starts to go

up towards V_{DD} . Since the back-to-back inverters form a positive feedback loop, regeneration occurs pulling $out-$ to ground and $out+$ to V_{DD} , as shown in Fig. 23 (b). Since the outputs of the strong-arm latch are set to V_{DD} during the reset phase, the RS latch is added to keep the generated decision for the SAR logic to pickup it up during the reset phase. This is done by turning OFF M5 and M6. The back-to-back inverters of the RS latch are weak enough for M5 and M6 to change their stored values when the new outputs are regenerated by the strong-arm latch. Finally, the XOR gate produces the ready signal by generating a high only when the regeneration process makes the output nodes of the strong-arm different (one of them is 1 and the other is 0), signaling that the comparison process is finished.

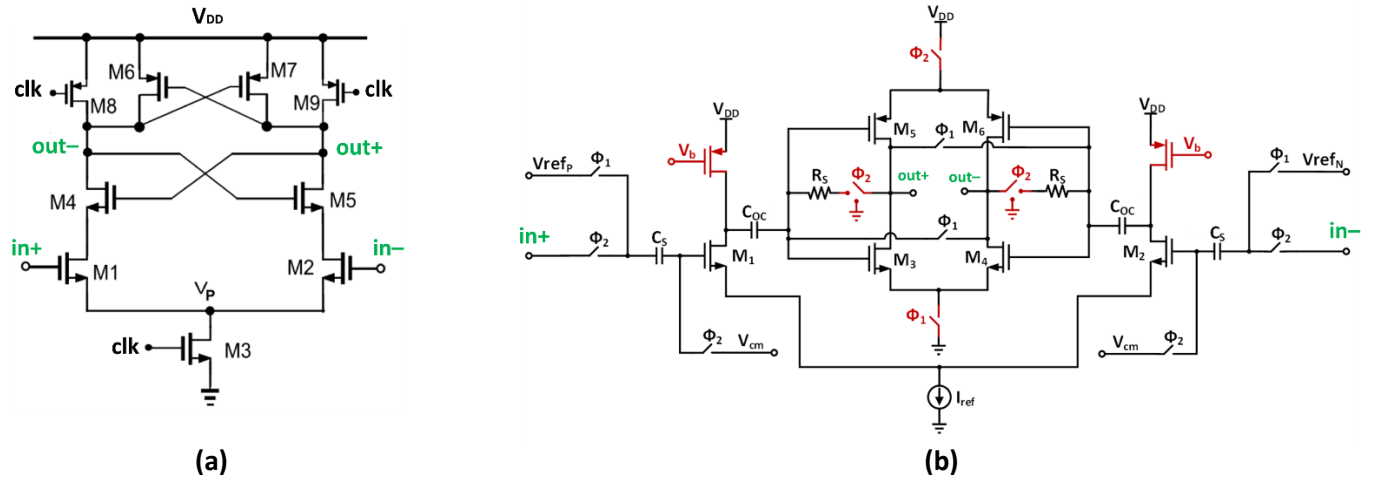


Fig.21. Types of Dynamic Comparator (a) Strong-Arm Comparator, (b) Two-Stage Comparator

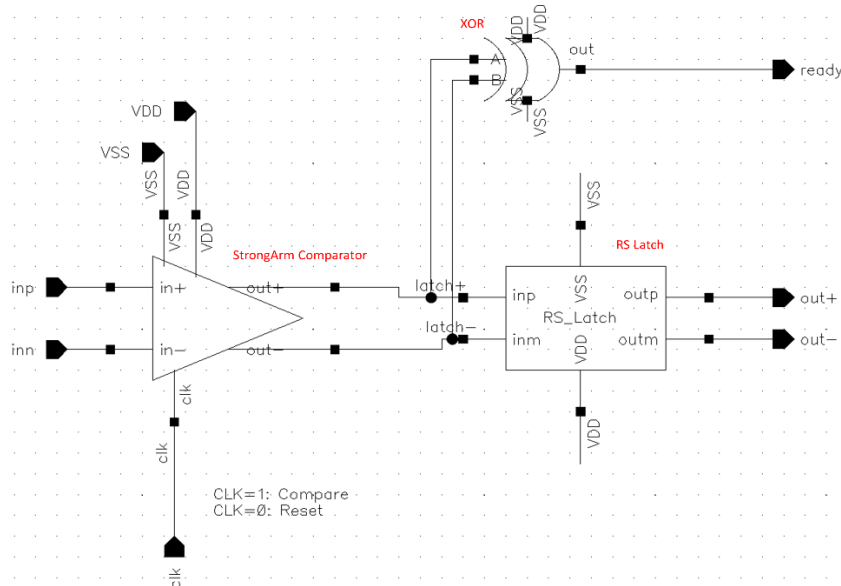


Fig.22. Block diagram of the proposed dynamic comparator

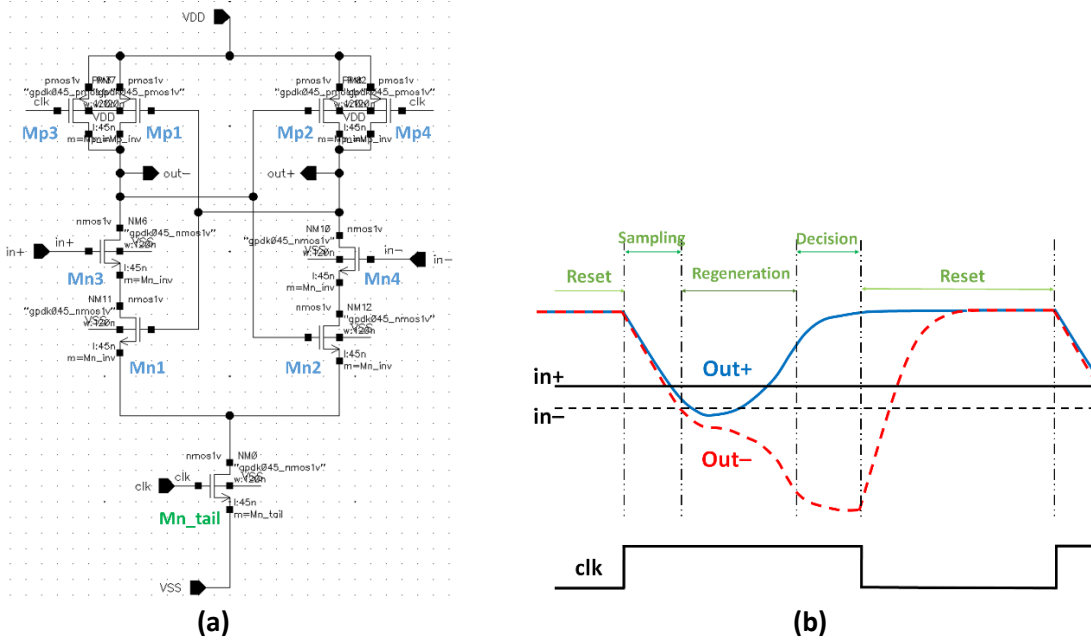


Fig.23. Proposed Strong-Arm Comparator (a) Schematics, (b) Operating phases [13]

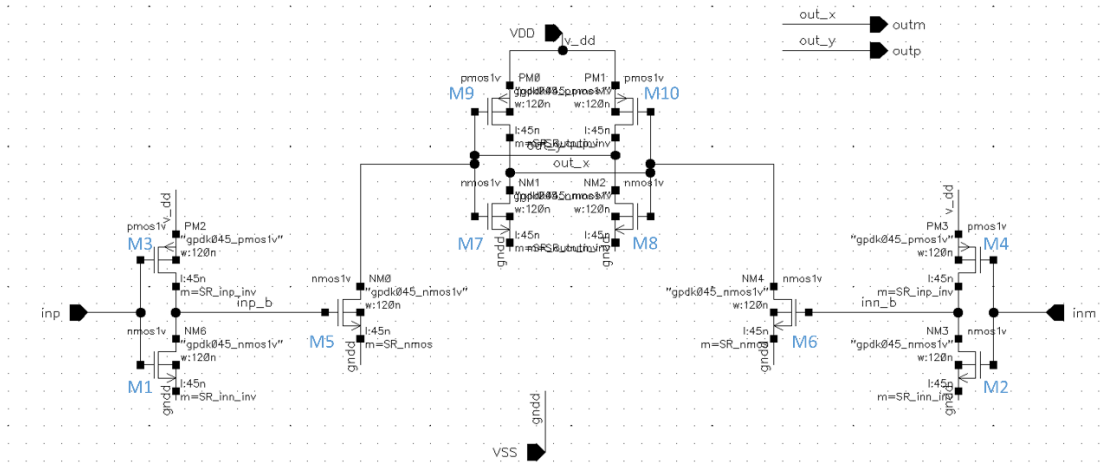


Fig.24. Schematics of the RS Latch

3.5 SAR logic

The proposed SAR logic shown in Fig. 25 is based on the SAR logic used for a synchronous SAR ADC [14]. The SAR logic block consists of 2 rows of D-flipflops, the sequencer register and the code register. The sequencer register keeps track of the sequence by keeping the output of one flipflop (FF) only equals to 1, while the outputs of the rest of the flipflops equal to 0. This 1 (or hot code) moves from one FF to the next at each rising edge of the “clk_SAR” (at each new iteration) and circulates through the register like how a one-hot-code ring counter operates. The hot code sets the bit being compared to 1, by triggering the “set” input of the corresponding FF in the code register. When the “set” of a FF in the code is triggered,

it outputs a 1 for its corresponding bit and also triggers the clock for the previous FF so that the previous bit would take the value of the comparator's decision. At the end of the conversion process, the code register holds the final DAC code and an EOC signals the output stage to take that code as the result of the ADC conversion.

Fig. 26 shows the schematics of the proposed SAR logic. The state table of the SAR logic block showing the whole 8-bit conversion process is presented in Table 1. The clock driving this block is “clk_SAR” coming from the internal clock generator, and the reset pin of this block is connected to “clk_sample” so that the outputs of all the FFs are reset during the sampling phase. For the 8-bit system, there are 8 main FFs in each of the sequencer and the code registers, in addition to 1 extra FF at beginning of the sequencer register to load the hot code during the sampling phase, and 2 extra FFs at the end of the registers to generate the EOC signal along with the final output code. The output of the comparator is connected to the D input of the main FFs of the code register. Since the capacitive DAC operates differentially, both the complementary outputs of the FFs are sent to the DAC.

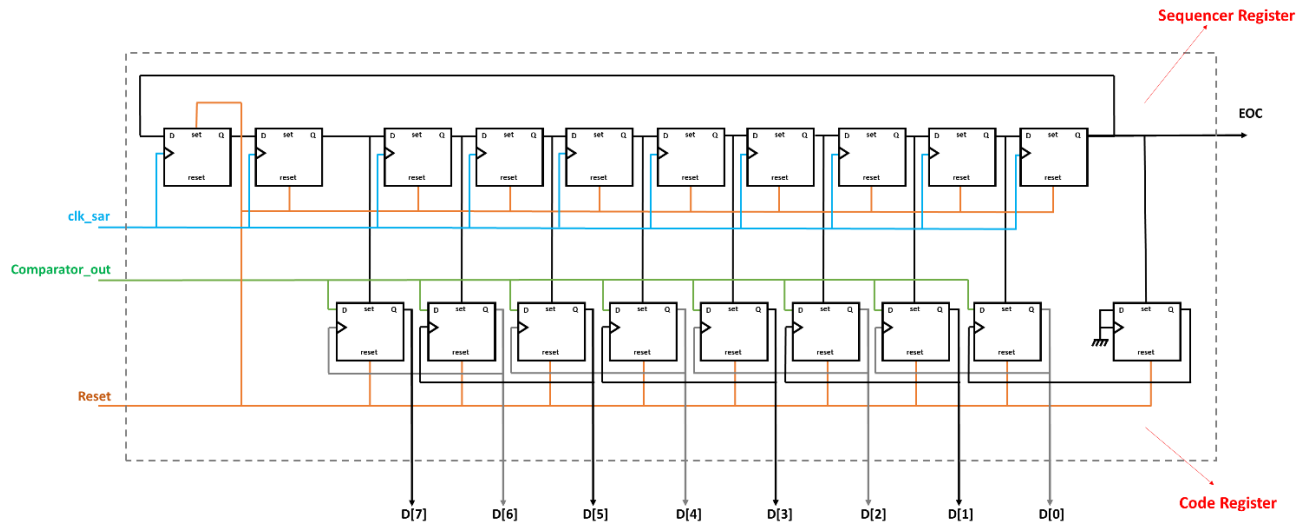


Fig.25. Block diagram of the SAR logic

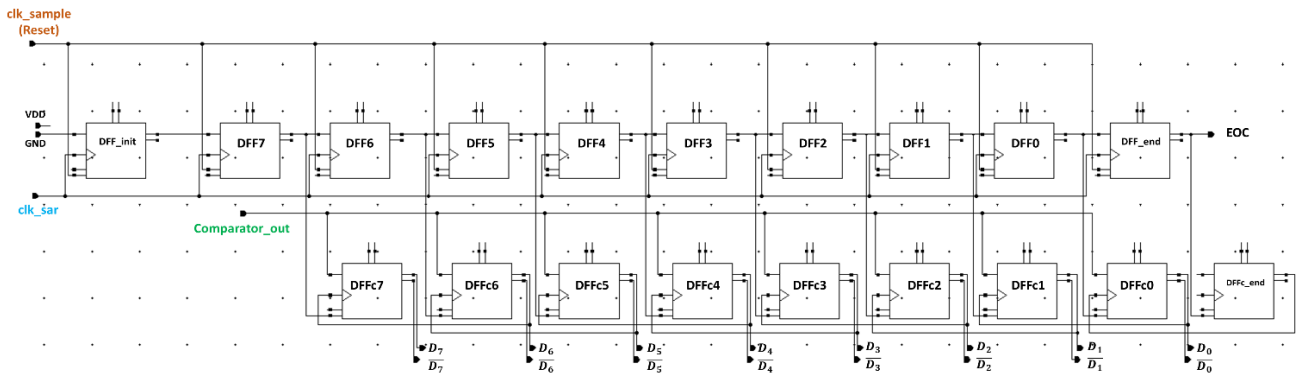


Fig.26. Schematics of the proposed SAR logic block

Table 1: SAR Logic States Table

		Sequencer Register Outputs										Code Register Outputs									Comparator Output
		DFF_init	DFF7	DFF6	DFF5	DFF4	DFF3	DFF2	DFF1	DFF0	DFF_end	DFFc7	DFFc6	DFFc5	DFFc4	DFFc3	DFFc2	DFFc1	DFFc0	DFFc_end	Comp_out
0	Sampling	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	X
1	Comparison 1	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	B7
2	Comparison 2	0	0	1	0	0	0	0	0	0	0	B7	1	0	0	0	0	0	0	0	B6
3	Comparison 3	0	0	0	1	0	0	0	0	0	0	B7	B6	1	0	0	0	0	0	0	B5
4	Comparison 4	0	0	0	0	1	0	0	0	0	0	B7	B6	B5	1	0	0	0	0	0	B4
5	Comparison 5	0	0	0	0	0	1	0	0	0	0	B7	B6	B5	B4	1	0	0	0	0	B3
6	Comparison 6	0	0	0	0	0	0	1	0	0	0	B7	B6	B5	B4	B3	1	0	0	0	B2
7	Comparison 7	0	0	0	0	0	0	0	1	0	0	B7	B6	B5	B4	B3	B2	1	0	0	B1
8	Comparison 8	0	0	0	0	0	0	0	0	1	0	B7	B6	B5	B4	B3	B2	B1	1	0	B0
9	Final Output	0	0	0	0	0	0	0	0	0	1	B7	B6	B5	B4	B3	B2	B1	B0	1	X

EOC

All the FFs in the SAR logic block are implemented using the true single-phase clock (TSPC) circuit shown in Fig. 27. TSPC FFs provide higher speeds, lower power consumption, and smaller area compared to the CMOS FFs [15]. Transistors M9, M11, and M12 are added to the conventional TSPC FF to add asynchronous set and reset capabilities. Inverters are added as an output stage to the TSPC FF to give strong rail-to-rail outputs.

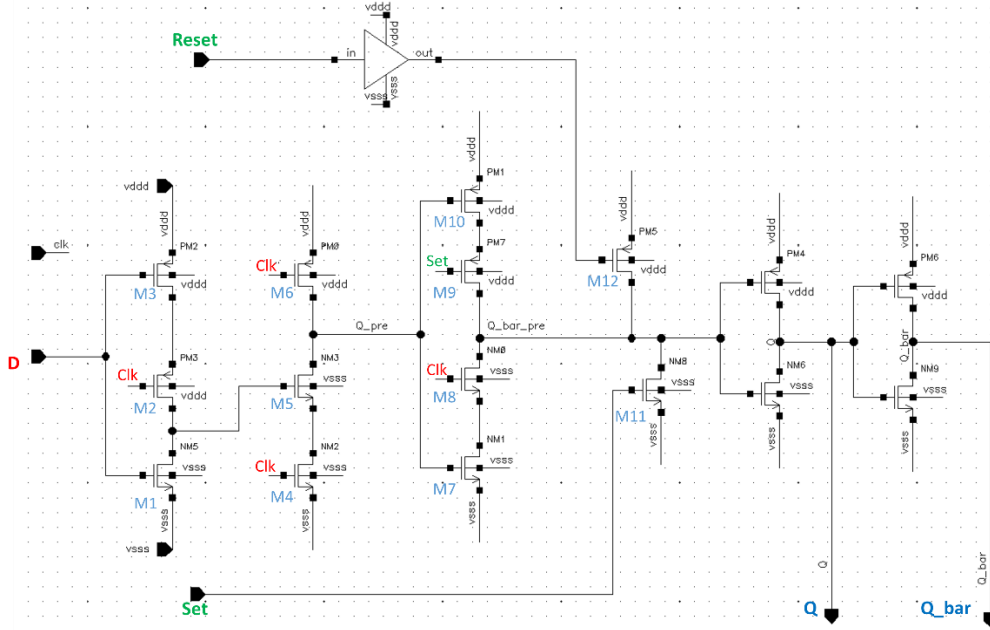


Fig.27. Schematics of the proposed TSPC circuit

3.6 Output Stage

To evaluate the performance of the SAR ADC, an output stage is added to the system to convert the digital output into an equivalent analog signal. The analog output is then compared to the ideal expected output which should be equivalent to the input signal. The output stage consists of 2 blocks, the output register and the output DAC.

3.6.1 Output Register

The output register's job is to receive the final DAC code at the rising edge of the EOC signal and releases it at the falling edge of EOC which happens at the rising edge of the next external clock cycle. The FFs used in this block are CMOS FFs as shown in Fig. 29. CMOS FFs are used in this block instead of TSPC FFs to ensure rail-to-rail node voltages within the circuit since the EOC signal is a low-frequency signal.

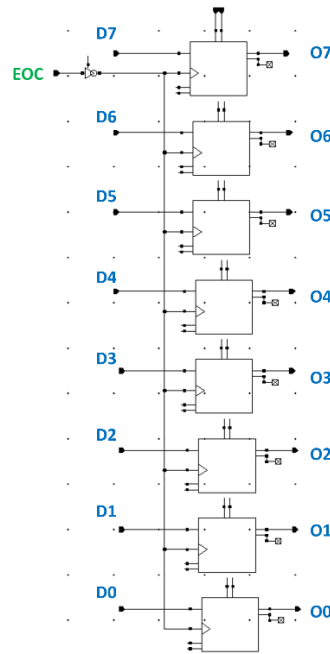


Fig.28. Schematics of the output register

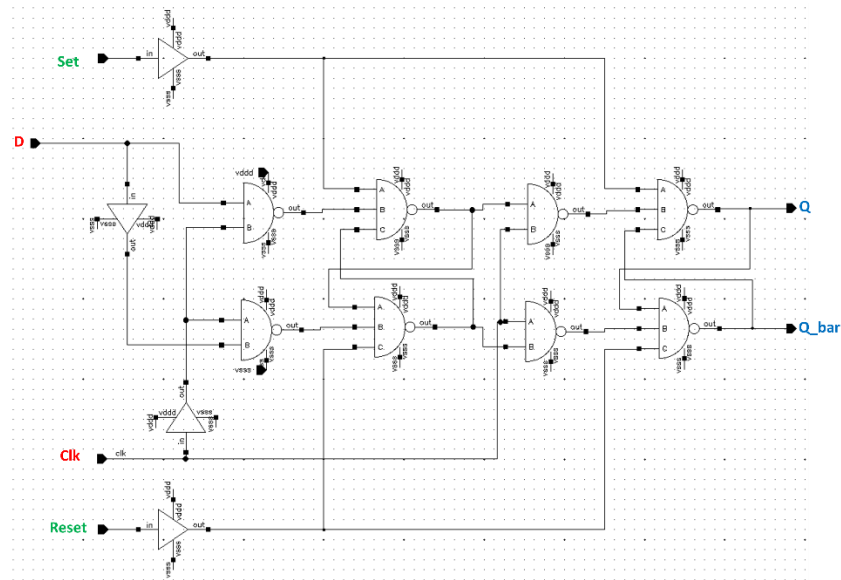


Fig.29. Schematics of the CMOS flipflop

3.6.2 Output DAC

The ideal output DAC used for evaluation is shown in Fig. 30. This block consists of 8 binary-weighted ideal voltage-controlled voltage sources (VCCS). They are stacked on top of each other to be summed together to represent the analog output given by equation 15. The final VCCS at the right is to correct the normalized analog output by multiplying by the right gain equal to the peak-to-peak swing of the input signal and shift the minimum level to V_{Refn} of the system.

$$V_{OUT} = \frac{1}{2^8} \cdot (D_7 \cdot 2^7 + D_6 \cdot 2^6 + D_5 \cdot 2^5 + D_4 \cdot 2^4 + D_3 \cdot 2^3 + D_2 \cdot 2^2 + D_1 \cdot 2^1 + D_0 \cdot 2^0) \quad (15)$$

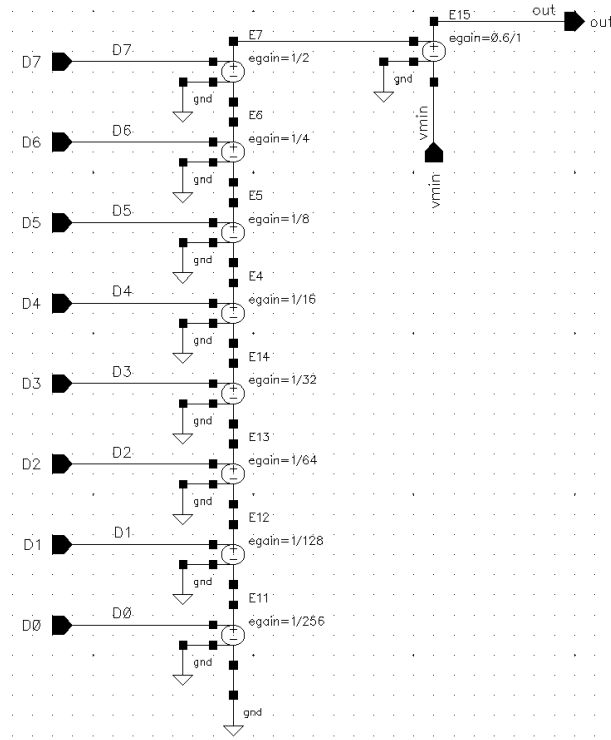


Fig.30. Schematics of the ideal output DAC

4. Results and Discussion

The first stage of the design is to build SAR ADC models using ideal components and VerilogA blocks to verify the functionality of the proposed system. Then in the second stage, the individual blocks are built and tested separately to ensure each block works properly individually. Then in the final stage of this project, the whole system with the actual blocks is simulated and evaluated. All the design work and simulations were done using Cadence Virtuoso analog design environment (ADE) software .

4.1 Ideal Models

To gain an understanding on the SAR ADC system, both the synchronous and the asynchronous SAR ADCs were modelled using the VerilogA blocks and the ideal components shown in appendix A. Fig. 31 shows the testbench for the 8-bit synchronous SAR ADC. The stimuli used were a clock of 100 MHz and an input of $\frac{7}{64} \times 10$ MHz. The output produced is of 10 MS/sec as shown in Fig. 32, since each 10 cycles of the clock produces 1 sample. Fig. 33 shows a zoomed view into 1 conversion cycle that contains 1 cycle for sampling, 8 cycles for bit comparisons, and 1 cycle to produce the final code.

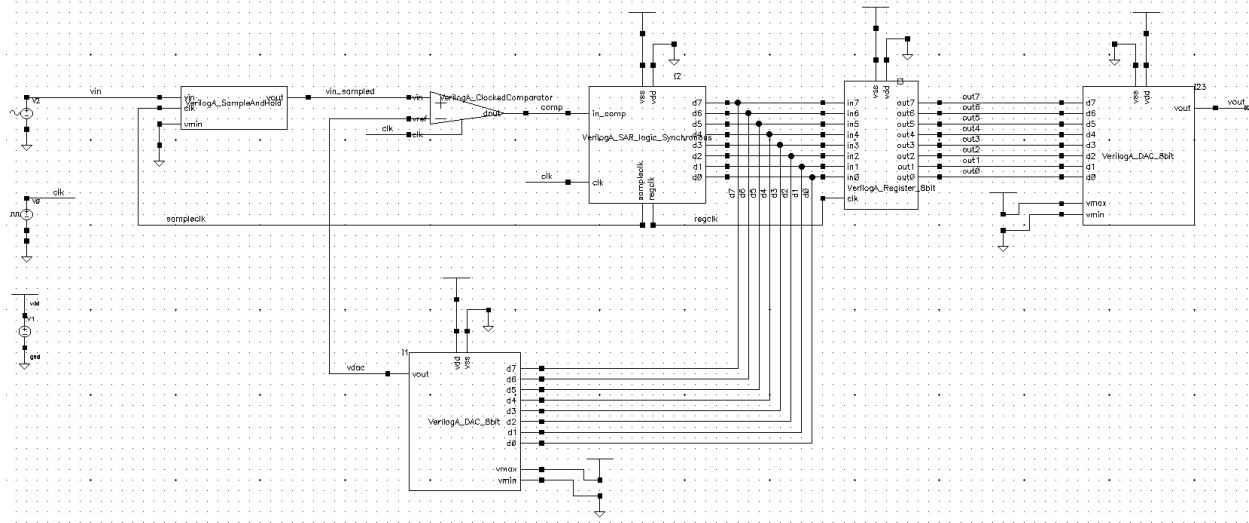


Fig.31. Testbench for the synchronous SAR ADC

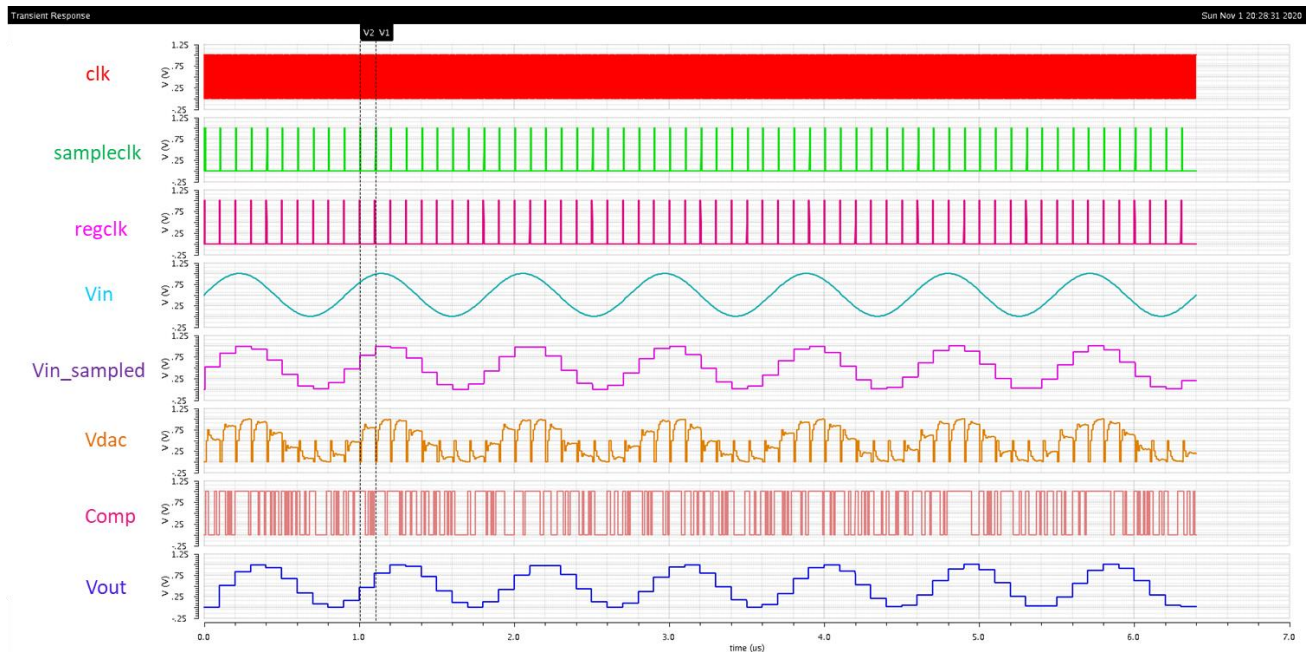


Fig.32. Transient Waveforms of the ideal synchronous SAR ADC testbench

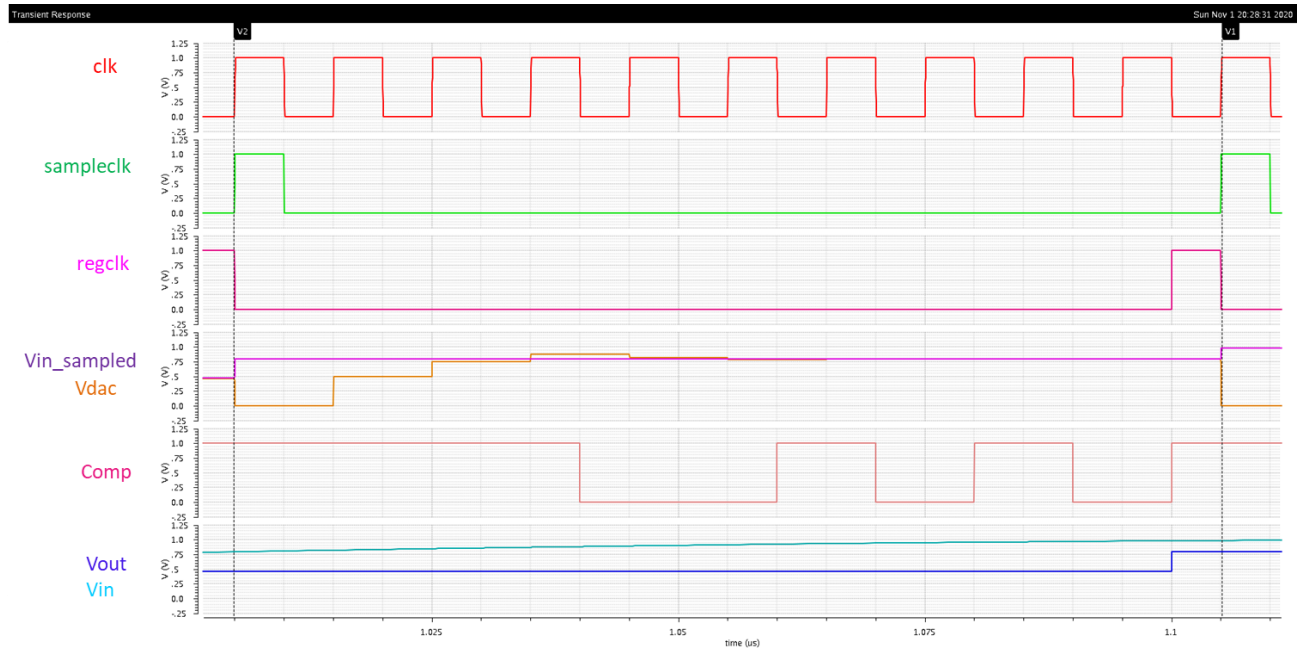


Fig.33. 1 conversion cycle for the synchronous SAR ADC

The testbench is then modified to test the 8-bit asynchronous SAR ADC functionality as shown in Fig. 34. The stimuli used were an external clock of 10 MHz and an input of $\frac{7}{64} \times 10$ MHz. Again, the output produced is of 10 MS/sec as shown in Fig. 35, and Fig. 36 shows the behavior within 1 conversion cycle that occurs in 1 external clock cycle. Since ideal components can change instantaneously, suitable ideal delay cells must be added inside the internal clock generator block and at the output of the comparator.

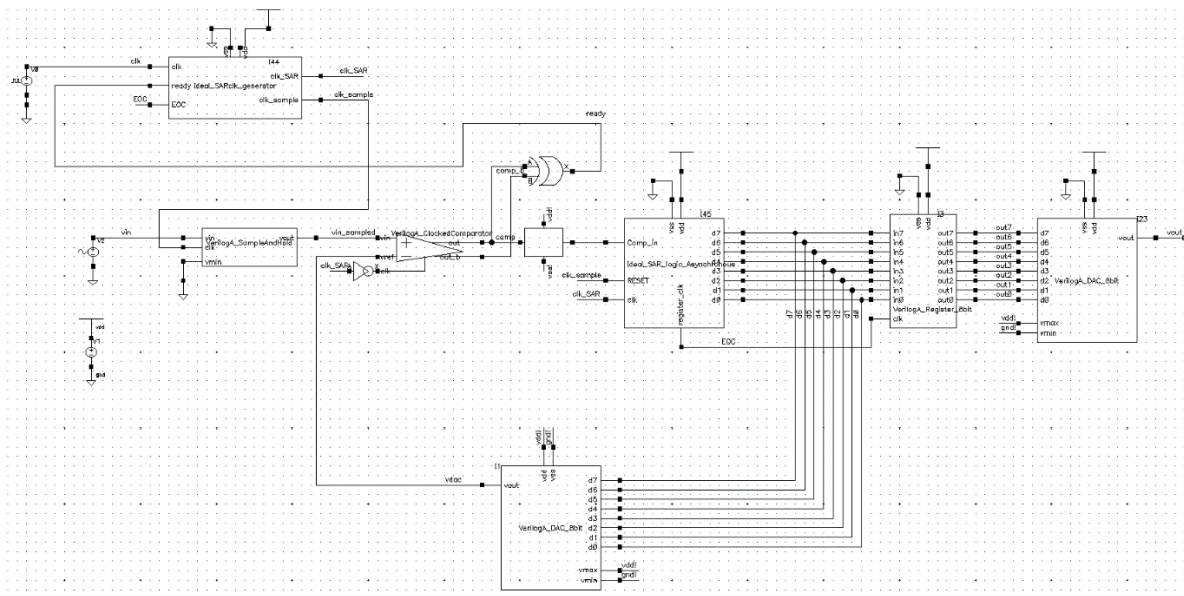


Fig.34. Testbench for the asynchronous SAR ADC

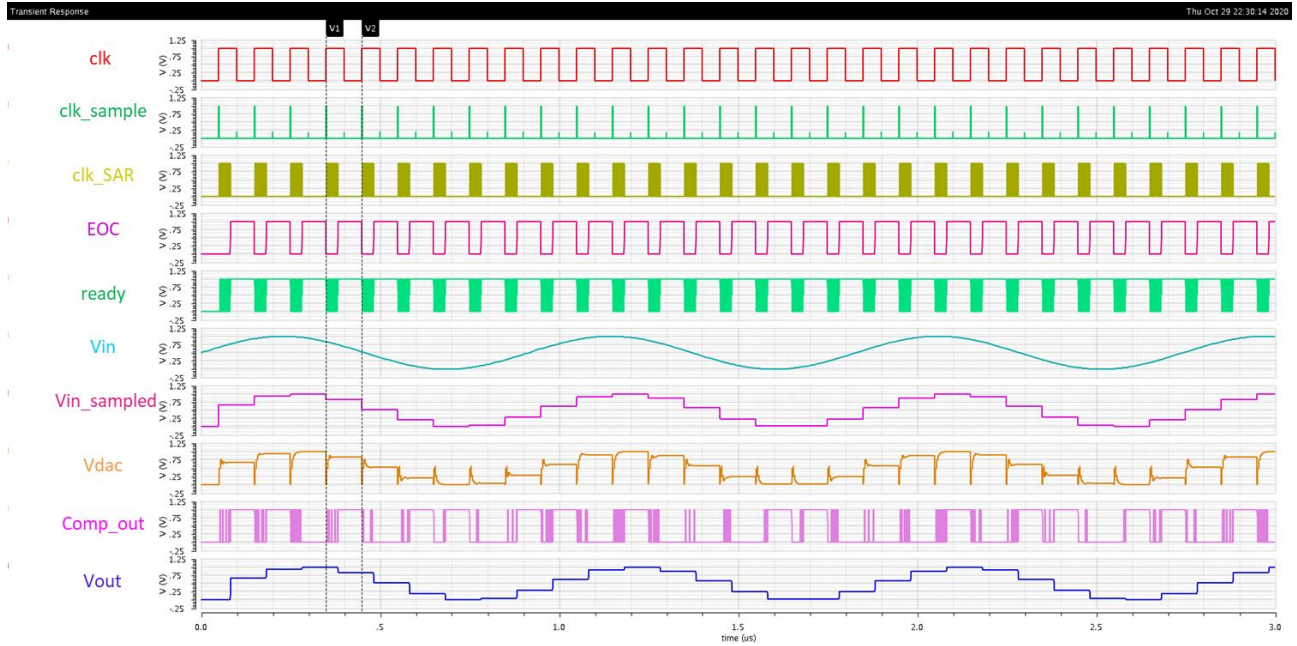


Fig.35. Transient Waveforms of the ideal asynchronous SAR ADC testbench

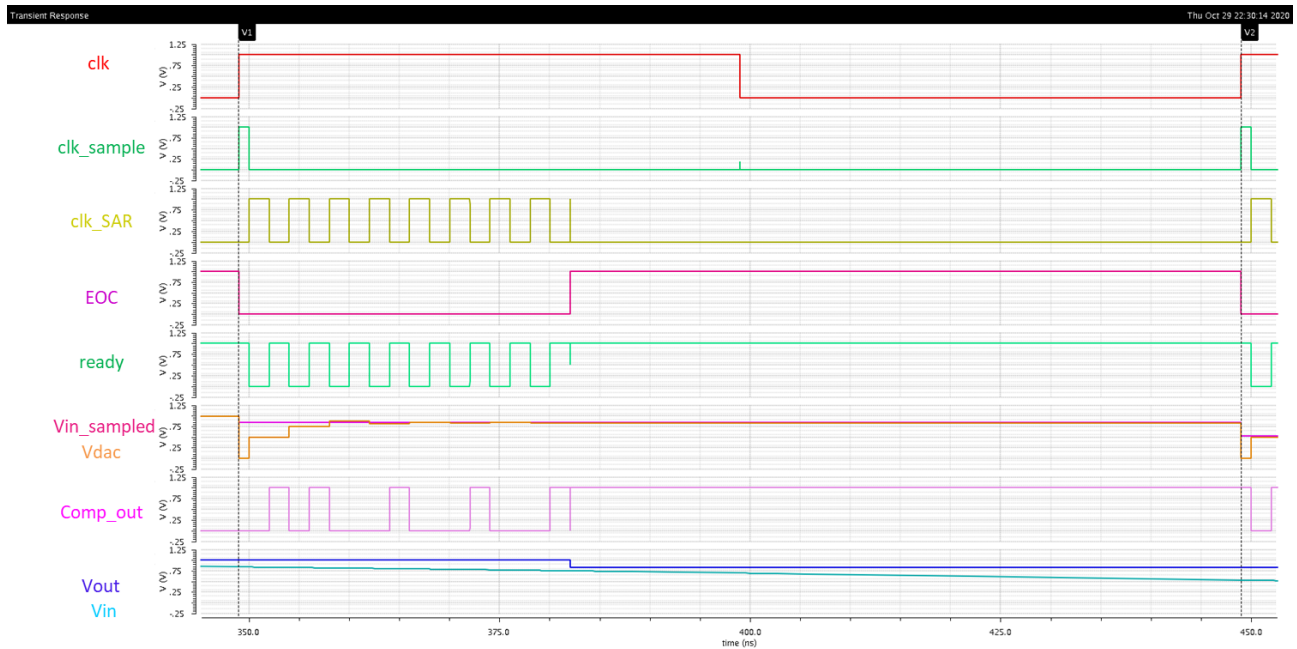


Fig.36. 1 conversion cycle for the asynchronous SAR ADC

4.2 Individual Blocks

The performance of the analog blocks are discussed in this section. For the digital blocks, the sizes of devices are chosen to keep the signal nodes with rail-to-rail voltage levels and with fast transition times in the presence of the loading effect from the following stages.

4.2.1 Bootstrapped Switch

To test the performance of the bootstrapped circuit, the output node is connected to all the capacitors in the DAC array as shown in Fig. 37. The sampling pulse is dictated by the time the output node needs to fully track the input node during the tracking phase ($\text{clk} = 1$). In the asynchronous SAR ADC, this pulse is generated by the delay cells in part A of the internal clock generator block explained in section 3.1 (Fig. 10). The smaller this pulse gets, the more challenging the design of the bootstrapped switch becomes for the given output capacitance load.

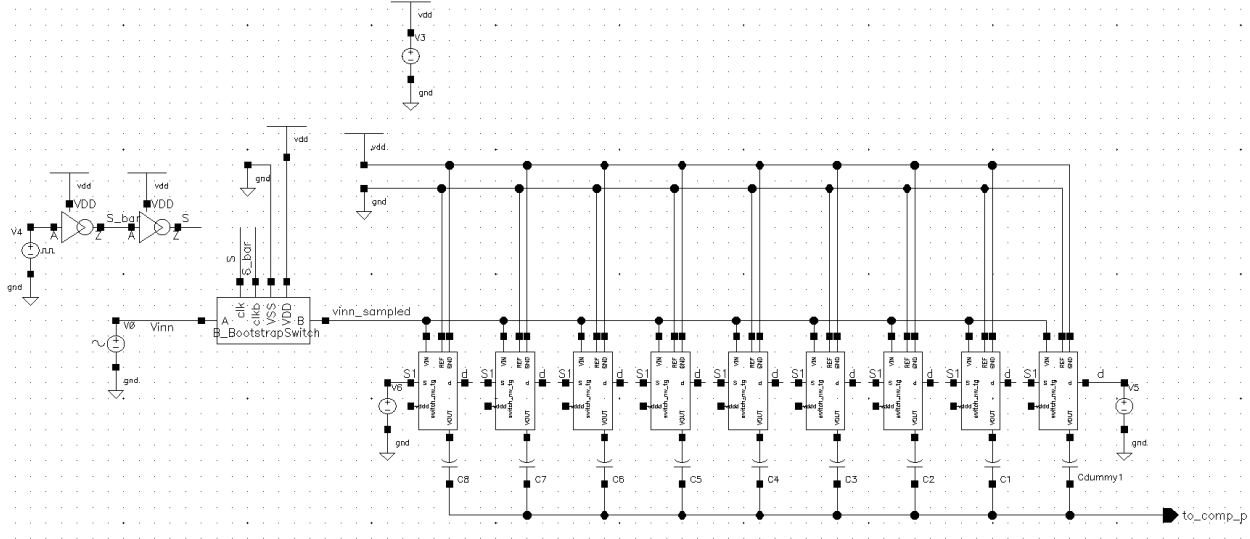


Fig.37. Testbench for the bootstrapped switch circuit

For the asynchronous SAR ADC to operate with a sampling pulse of 3.125 ns, the testbench parameters are determined as follows:

$$f_{CLK} = f_{sampling} = \frac{1}{2 \times 3.125 \text{ ns}} = 160 \text{ MHz}$$

For FFT of $N = 64$, and to have 1 input cycle every 7 sample cycles, then:

$$f_{IN} = \frac{7}{64} \times 160 = 17.5 \text{ MHz}$$

$$T_{sampling} = \frac{1}{f_{sampling}} = 6.25 \text{ ns}$$

Then, the simulation time to capture 64 sampling cycles or 7 input cycles:

$$T_{simulation} = 64 \times 6.25 \text{ ns} = 400 \text{ ns}$$

And Adding 9ns as an initialization time, then the measurement time taken is:

$$T_{measurement}: 9\text{ns} \rightarrow 409\text{ns}$$

The waveforms of the bootstrapped switch's testbench in Fig. 39 and Fig. 40 show how the output “vinn_sampled” tracks the input when $\text{clk} = 1$ and holds the value when $\text{clk} = 0$. Using the setup in Fig. 41, the generated output spectrum in Fig. 42 gives an ENOB of 10.02 bits and SNR of 62.08 dB.

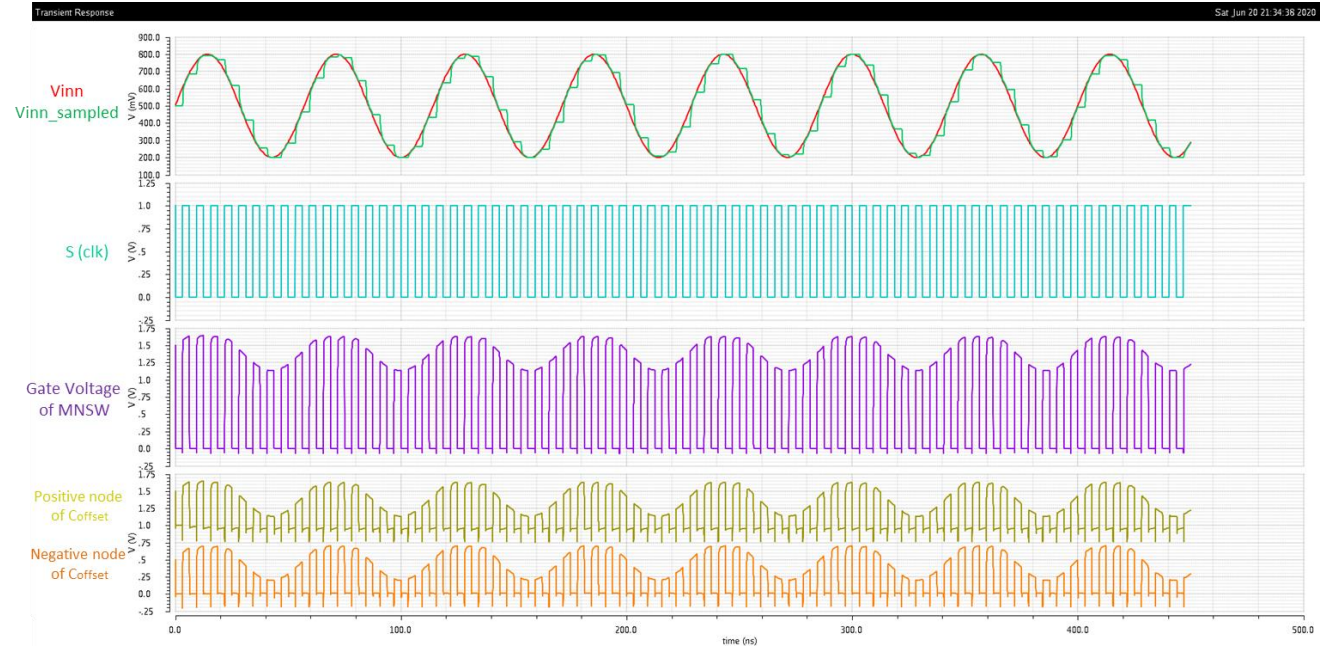


Fig.39. Transient Waveforms of the bootstrapped switch testbench

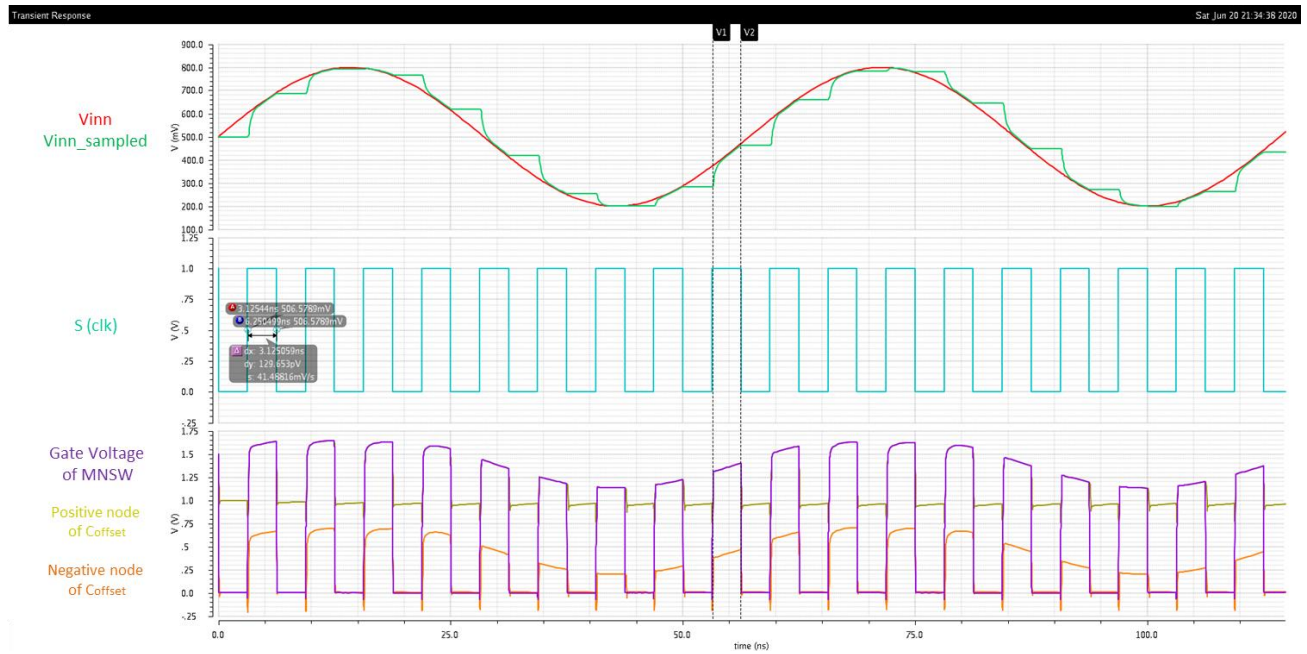


Fig.40. 1 cycle of the input to the bootstrapped switch

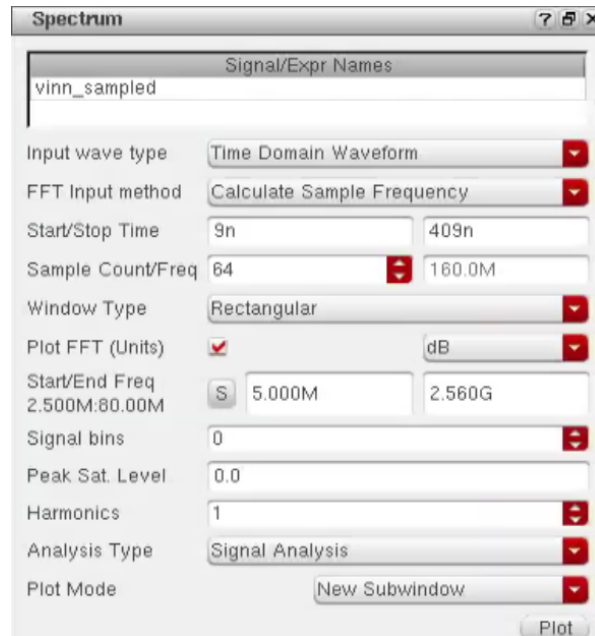


Fig.41. Spectrum measurement setup for the bootstrapped circuit's output signal

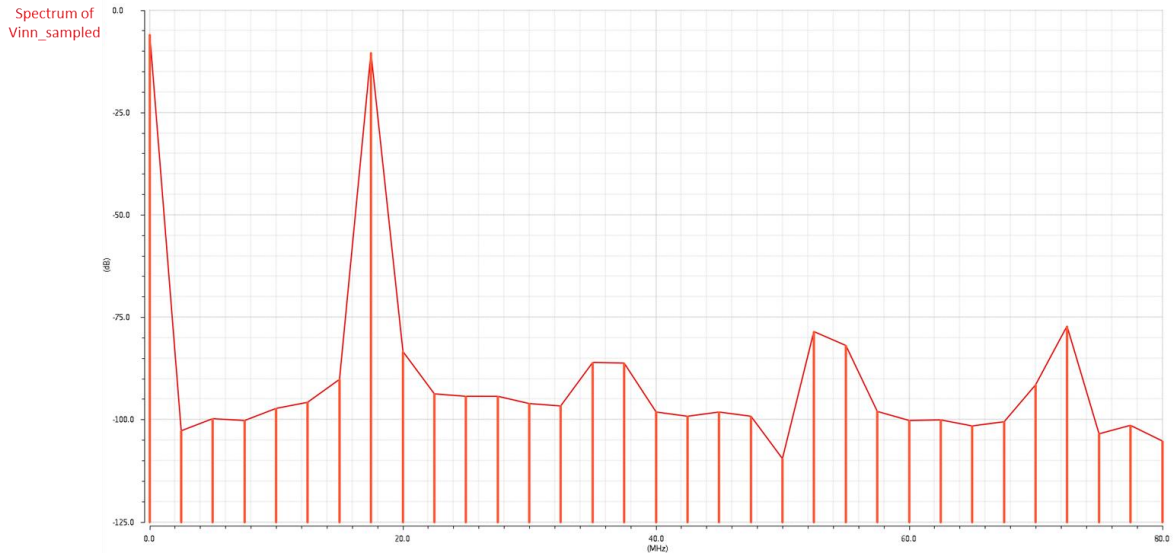


Fig.42. Spectrum of the sampled output of the bootstrapped switch

4.2.2 Capacitive DAC

One of the key parameters that determine the performance of the SAR ADC is the settling time of the capacitive DAC. The testbench in Fig. 43 is used to determine the effect of changing the value of the unit capacitor and the sizing of the transmission gate switches' devices. The settling time decreases as the capacitors' values decrease, but that leads to a lower SNR due to higher thermal noise. Another way to decrease the settling time is to make the transmission gate switches less resistive by increasing the W of the switch's transistors but increasing the W too much can lead to significant parasitic capacitance. Fig. 44 (a)

and (b) show the waveforms with faster settling and slower settling times respectively. The delay from “clk_SAR” to the reset input of the FF in part C of the internal clock generator in Fig. 10 is chosen to be longer than the settling time of the DAC. The proposed system is designed for a 2fF unit capacitance in the DAC and a settling time < 1ns.

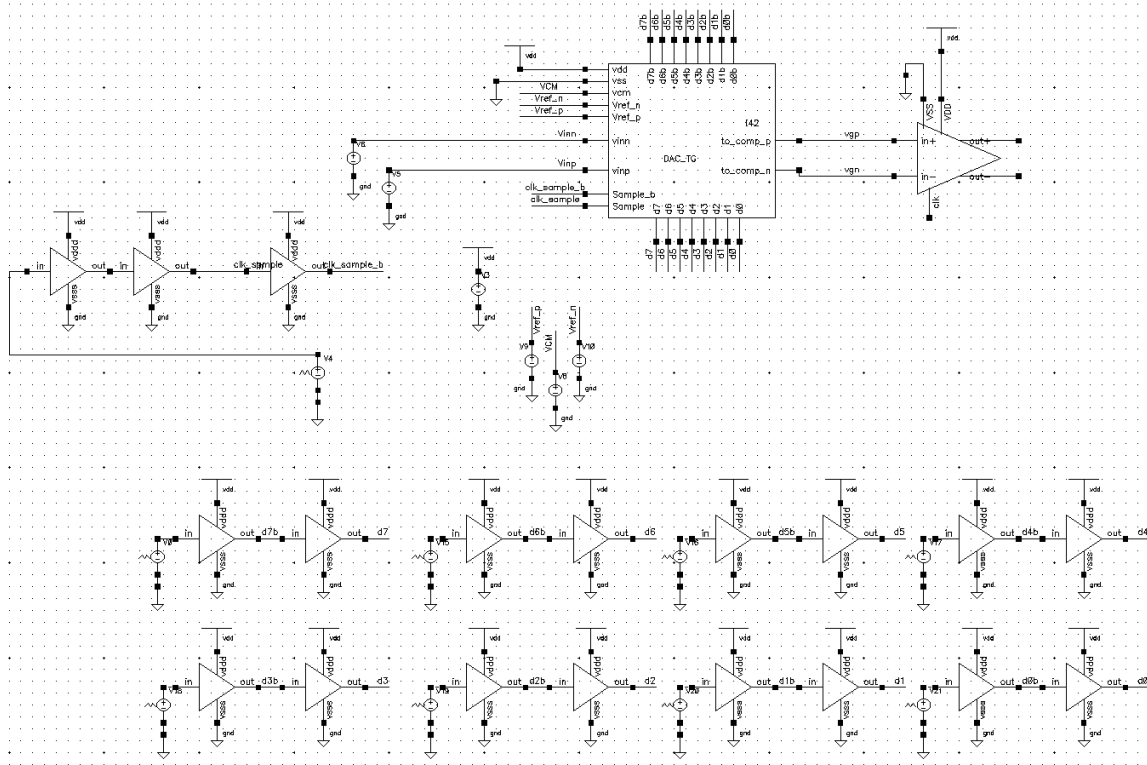
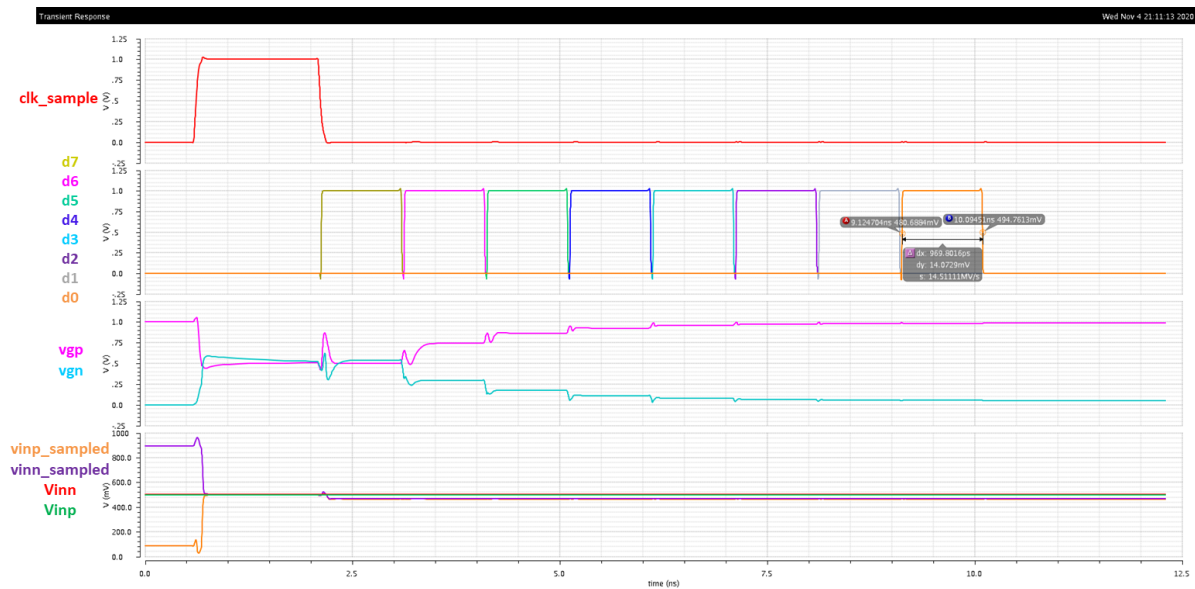
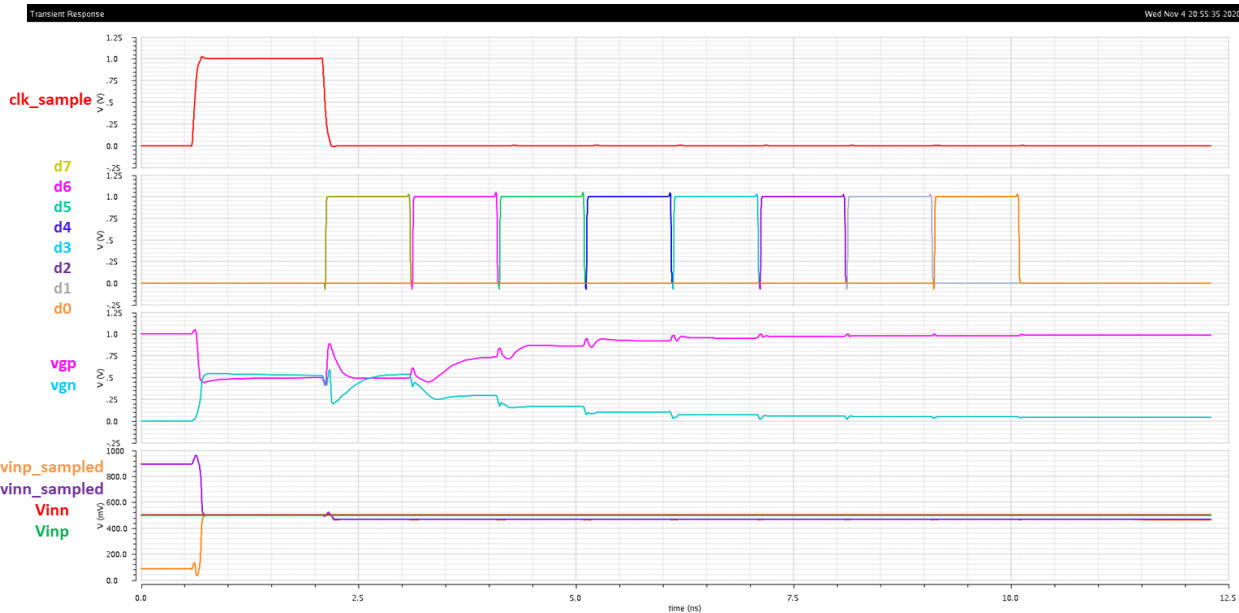


Fig.43. Testbench for the capacitive DAC



(a)



(b)

Fig.44. Transient waveforms of the capacitive DAC testbench

(a) with faster settling (b) with slower settling

4.2.3 Comparator

The resolution of the ADC is determined by the minimum input that the comparator can detect and resolve. The comparator must be able to resolve a differential input voltage of 1 LSB or less. For an 8-bit ADC and a single-ended input full-scale range of 0.6V, the $\frac{1}{2}$ LSB is equal to $\frac{1}{2} \cdot \frac{0.6}{2^8} = 1.17 \text{ mV}$. The resolution of the comparator is tested through the overdrive recovery test using the testbench in Fig. 45. In the overdrive test, the comparator is tested by changing the input signal between different levels and seeing if the comparator can recognize these changes within 1 clock cycle from the change. Fig. 46 and 47 show the overdrive test waveforms at different inputs. The inputs change during the comparator's reset phase ($S_{\text{bar}} = 0$) and the comparator starts the comparison at the rising edge of its clock. From the overdrive test, the minimum differential input that can be resolved by the designed comparator is 0.6 mV with a clock of 300 MHz.

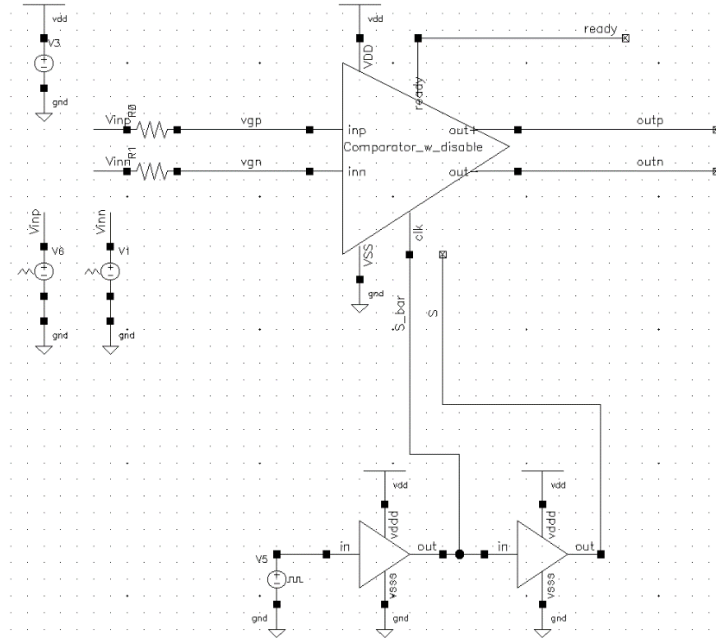


Fig.45. Testbench for the dynamic comparator

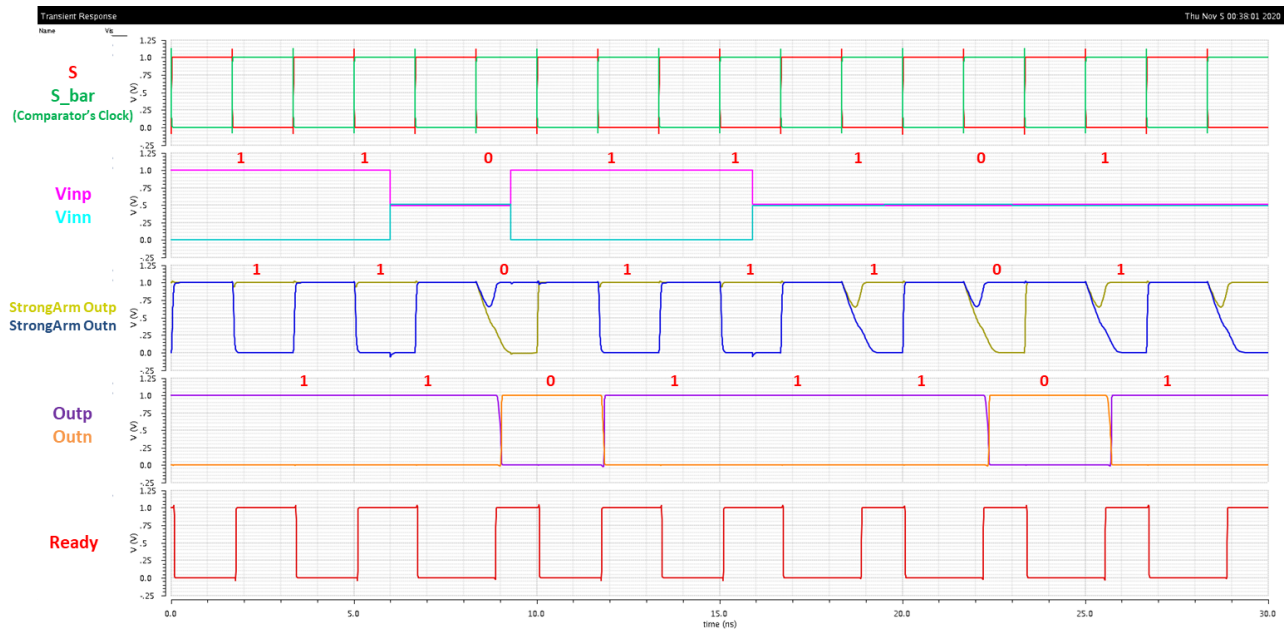


Fig.46. Overdrive recovery test ($V_{in} = 20\text{mV}$ and frequency = 300MHz)

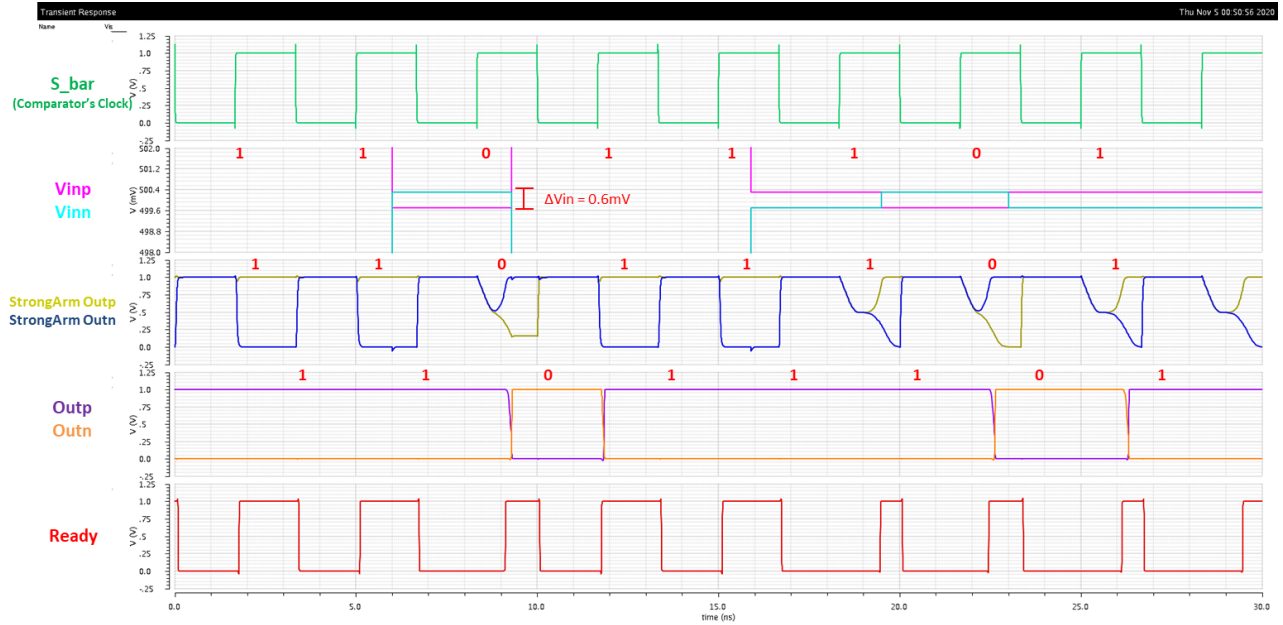


Fig.47. Overdrive recovery test ($V_{in} = 0.6\text{mV}$ and frequency = 300MHz)

4.3 Whole Asynchronous SAR ADC system

The testbench used to test the whole asynchronous SAR ADC is shown in Fig. 48, with all the important node voltages probed and plotted in Fig. 49. The stimuli used were an external clock of 20 MHz and inputs “Vinp” and “Vinn” each of frequency $\frac{7}{64} \times 20$ MHz and amplitude of 0.6 V (from 0.2 V to 0.8 V), producing an output of 20 MS/sec.

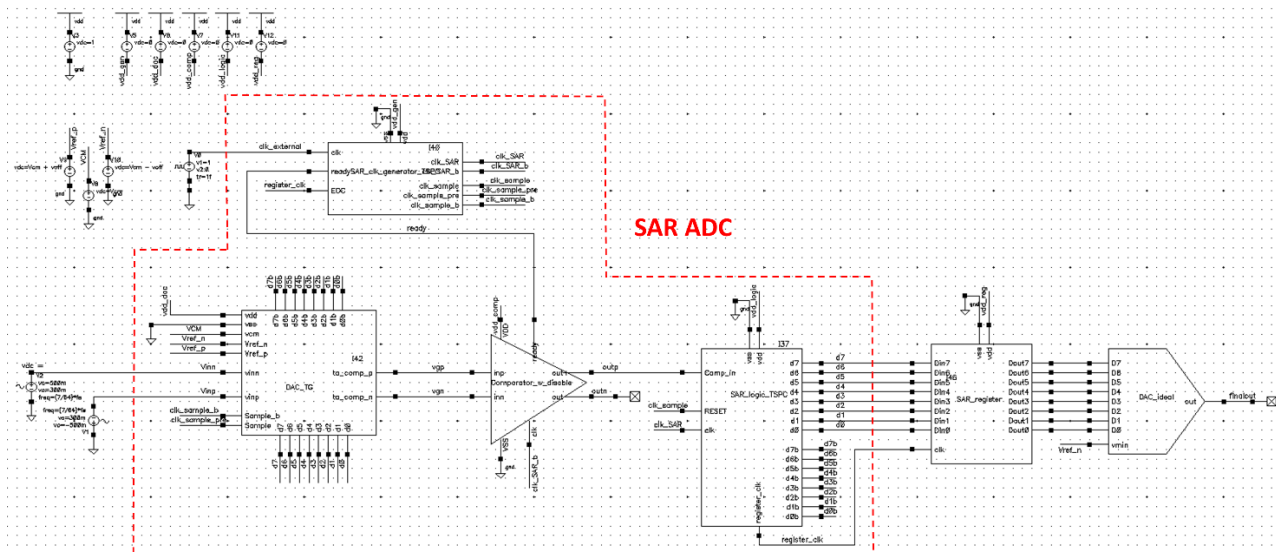


Fig.48. Testbench of the whole asynchronous SAR ADC

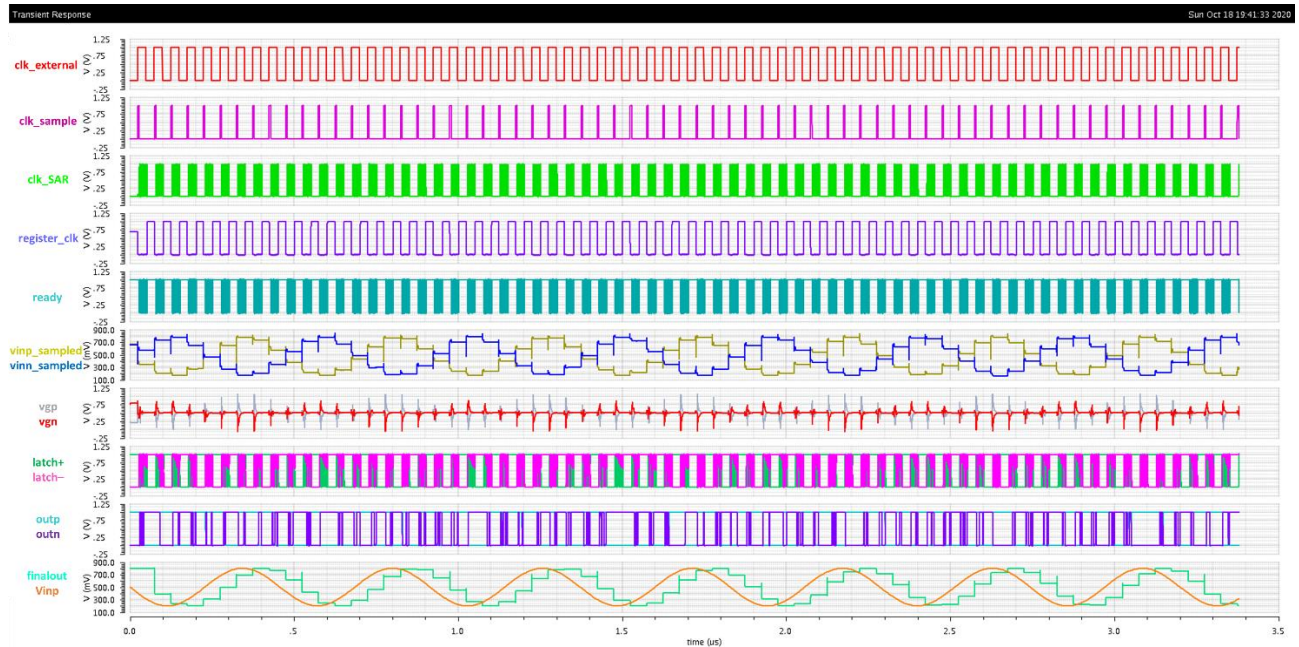


Fig.49. Transient waveforms of the asynchronous SAR ADC testbench

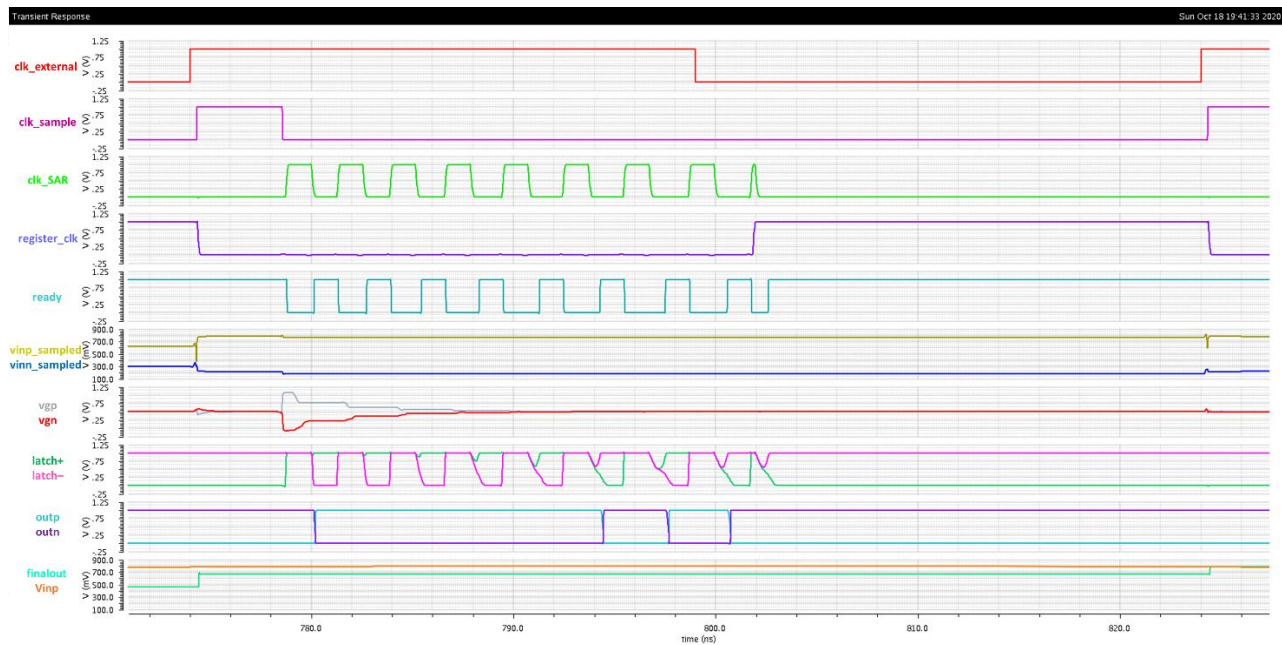


Fig.50. Waveforms of the asynchronous SAR ADC testbench during 1 conversion cycle

Fig. 50 is a zoomed snapshot of the waveforms during 1 conversion cycle. At the rising edge of “clk_external”, a pulse is generated at “clk_sample” in which the sampling of the inputs occurs. Once “clk_sample” goes low, “clk_SAR” goes high causing the code from the SAR logic block to redistribute the charges in the capacitive DAC, changing the input nodes to the comparator “vgp” and “vgn”. At the falling edge of “clk_SAR”, the DAC would have settled already, and the comparator is activated

(regeneration phase). When the outputs of the strongarm latch “latch+” and “latch–” give opposite rail-to-rail output levels (a strong high and a strong low), the “ready” signal goes high which triggers “clk_SAR” to go up high after a certain delay. After 8 cycles of “clk_SAR”, the EOC signal or “register_clk” signals the output register to output the result of the conversion process, which is the final digital code from the SAR logic block. To measure the performance, the digital output is converted to its analog equivalence and the FFT is taken for this analog output.

To produce the frequency response of the output of the system, the measurement setup in Fig. 51 is used. The stimuli and the measurement parameters are determined from the external clock frequency of the system as follows:

$$f_{CLK} = 20 \text{ MHz}$$

For FFT of $N = 64$, and to have 1 input cycle every 7 sample cycles, then:

$$f_{IN} = \frac{7}{64} \times 20 = 2.1875 \text{ MHz}$$

$$T_{\text{sampling}} = \frac{1}{f_{\text{sampling}}} = 50 \text{ ns}$$

Then, the simulation time to capture 64 sampling cycles or 7 input cycles:

$$T_{\text{simulation}} = 64 \times 50 \text{ ns} = 3200 \text{ ns}$$

And Adding 150 ns as an initialization time, then the measurement time taken is:

$$T_{\text{measurement}}: 150\text{ns} \rightarrow 3350\text{ns}$$

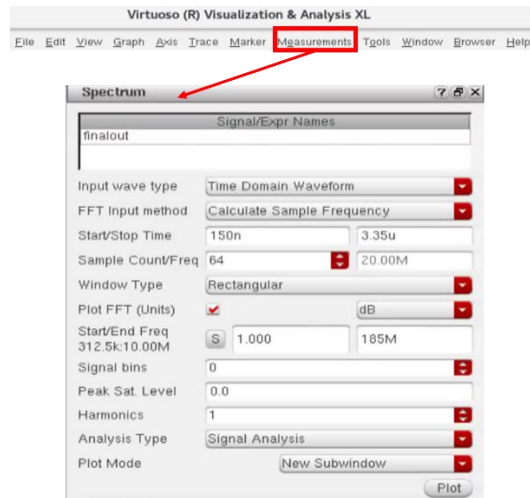


Fig.51. Spectrum measurement setup for the asynchronous SAR ADC output signal

The spectrum of the final output of the asynchronous SAR ADC testbench, from which the key dynamic ADC performance metrics are measured, is shown in Fig. 52. The signal-to-noise-and-distortion

ratio (SINAD) is calculated from the signal-to-noise ratio (SNR) and the total harmonic distortion (THD) as in equation (16). Since the THD is neglected in this setup, the SINAD and SNR are considered the same. The theoretical SNR for an N-bit ADC driven by a full-scale sine wave and that only considers the quantization noise is given by equation (17). Using the equation of SNR, the effective number of bits (ENOB) can be obtained from equation (19). The ENOB represents the actual resolution of the ADC after taking the noise and the distortion of the system into consideration. As a measure of the strength of the desired signal compared to the worst spur in the output spectrum, the spurious-free dynamic range (SFDR) is also found [16].

$$SINAD \text{ (in dBs)} = 10 \log \left(\frac{P_{Signal}}{P_{noise} + P_{distortion}} \right) = -10 \log \left(10^{\frac{-SNR}{10}} + 10^{\frac{-THD}{10}} \right) \quad (16)$$

$$SNR \text{ (in dBs)} = 10 \log \left(\frac{P_{Signal}}{P_{noise}} \right) = 6.02 N + 1.76 \quad (17)$$

$$THD \text{ (in dBs)} = 10 \log \left(\frac{P_{Signal}}{P_{distortion}} \right) \quad (18)$$

$$ENOB \text{ (in Bits)} = \frac{SINAD - 1.76}{6.02} \quad (19)$$

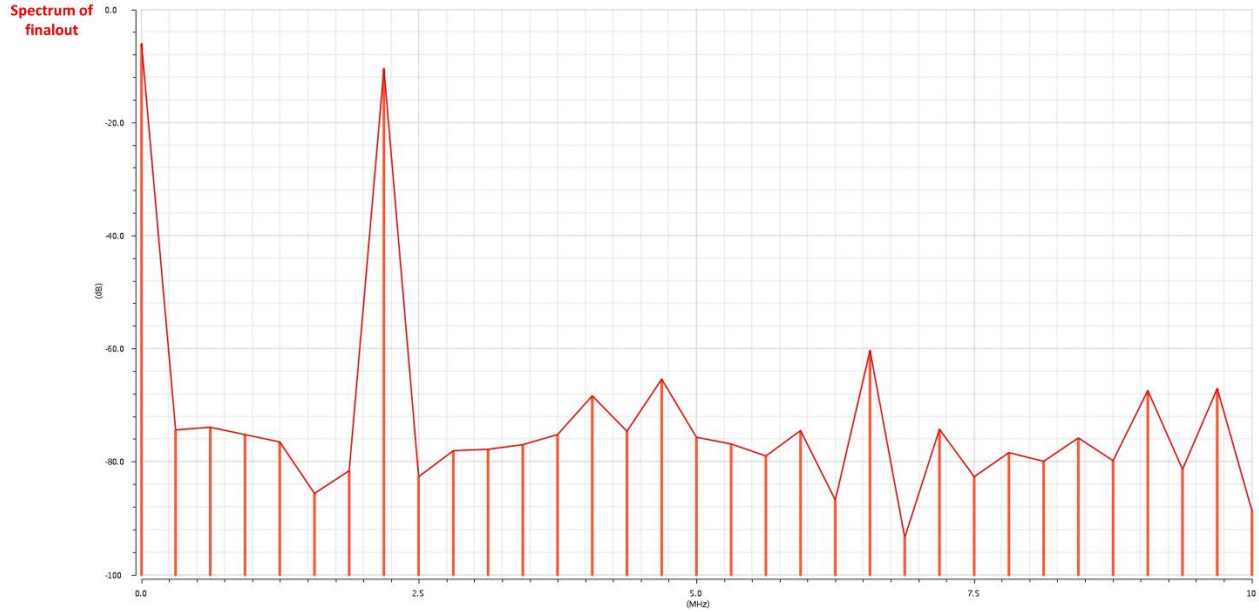


Fig.52. Spectrum of the output of the asynchronous SAR ADC system

These key metrics and parameters of the system are displayed in Table 2. The figure-of-merit (FOM) of the system is defined by equation (20). The aim of the design is to decrease the FOM, which mainly involves decreasing the total power consumption and improving the ENOB. The power is improved by using circuits that consume no static power like the strongarm latch and the TSPC flipflops, while the

ENOB is improved by using techniques like bootstrapped switching and highly sensitive strongarm latch design. The achieved FOM from the proposed SAR ADC is 15 fJ/conversion-step which is comparable to earlier asynchronous SAR ADC designs, as shown in Ref [4].

$$FOM = \frac{P_{total}}{2^{ENOB} \cdot f_s} \quad (20)$$

Table 2: Performance Summary

Parameter	Value
Technology	45 nm CMOS
Power Supply	1 V
Resolution	8 bits
Sampling Rate	20 MS/s
Common-Mode Voltage	0.5 V
Differential Input Range	1.2 V
Sampling Unit Cap	2 fF
SINAD	46.00 dB
SFDR	49.91 dB
ENOB	7.35 bits
Total Power	49.124 uW
FOM	15.06 fJ/conversion-step

Table 3 and Fig. 53 show how the total power is divided among the different blocks. Most of the power consumption come from the digital blocks, with the delay cells being the biggest contributor to the high power. The power of the digital circuits usually scales down well with supply, since the dynamic power is directly proportional to V_{DD}^2 as shown in equation (21). This allows the proposed asynchronous SAR ADC to have the potential to offer more power reduction in future design work.

$$P_{dynamic} = \alpha \cdot C_L \cdot V_{DD}^2 \cdot f_{clk} \quad (21)$$

Table 3: Power Consumption Per Block

Block	Power
Internal Clock Generator (including delay cells)	35.33 uW
SAR Logic	7.480 uW
Comparator	5.216 uW
DAC + Switches	1.099 uW
Total Power	49.124 uW

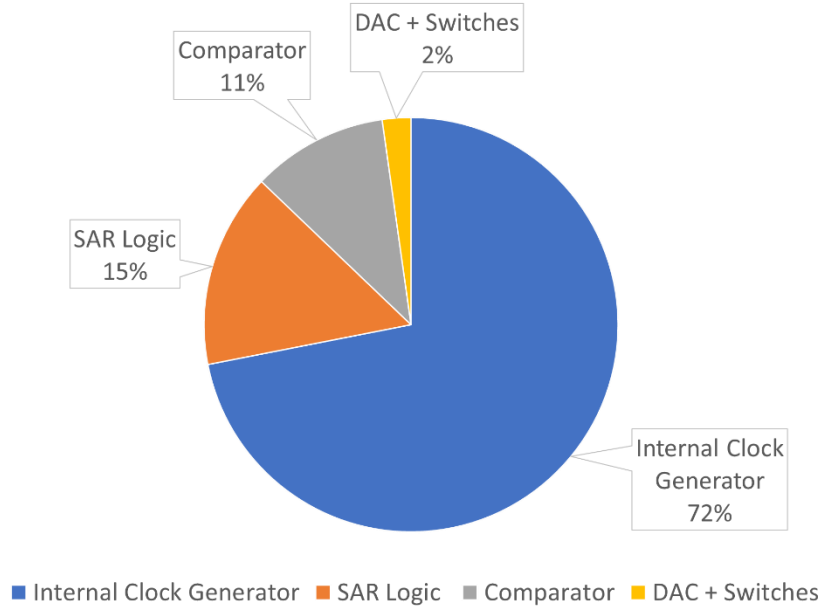


Fig.53. Power consumption percentage per block

5. Conclusion

A low-power 8-bit asynchronous SAR ADC, with a clock input of 20MHz and designed using CMOS 45nm technology at 1 V, is presented. The design uses an internal clock generator that controls the conversion process by generating clock signals for the rest of the circuit. Bootstrapped switches are used to sample the differential input signal and to provide high ENOB. A binary-weighted charge-redistribution capacitive DAC provides the input levels to be compared by the comparator while consuming no static power. The comparison is done by a low-power strongarm-based comparator with a resolution of 0.6 mV. The SAR logic block that generates the digital code uses TSPC flipflops to further reduce power consumption. The measured ENOB of the system is 7.35 bits while consuming a total power of 49.12 uW, resulting in an FOM of 15 fJ/conversion-step.

Improvements can be further applied to the design to get better FOM. As seen in the results section, most of the power consumption comes from the digital blocks, so power reduction is possible by lowering the supply voltage for these blocks or by using other logic topologies, like dynamic CMOS logic. Also, other capacitive DAC topologies offer lower switching energy through split-capacitor arrangements and monotonic switching methods, as discussed in Ref [9]. Changing the switching scheme in the DAC will require a re-design of the SAR logic block as well. Other topologies for the internal clock generator can also help in saving power, as discussed in Ref [6]. The proposed design shows that the asynchronous SAR ADC architecture can provide an energy and FOM efficient ADC without the use of an oversampled clock.

Appendix A: Verilog-A codes and Ideal models

In this section, the VerilogA codes used in the ideal models for the synchronous and the asynchronous SAR ADCs are presented.

A.1: Synchronous SAR ADC:

A.1.1 Sample-And-Hold:

```
// VerilogA for SampleAndHold
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_SampleAndHold(clk,vin,vmin,vout);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;      //vdd is 1v

input clk,vin,vmin;
output vout;

electrical vout,vin,vmin,clk;
real v;

analog begin

// Sampling Phase (+1 is for rising edge, -1 is for falling edge)
  @(cross(V(clk) - clk_threshold, +1))
    v = V(vin);

  V(vout) <+ transition(v,delay,ttime);
end
endmodule
```

A.1.2 Clocked Comparator:

```
// VerilogA for ClockedComparator
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_ClockedComparator(dout,vref,vin,clk);

parameter real clk_th=0.5;
parameter real delay = 0;
parameter real ttime = 1p;

input vin,vref,clk;
output dout;

electrical dout,vref,vin,clk;
real d_result;

analog begin

  @(cross(V(clk) - clk_th, -1)) begin
    if(V(vin) > V(vref)) begin
      d_result = 1;
    end
  end
end
```



```

        else begin
            d_result = 0;
        end
    end
end

V(dout) <+ transition(d_result,delay,ttime);
end
endmodule

```

A.1.3 SAR Logic:

```

// VerilogA for SAR_VerilogA, VerilogA_SAR_logic_Synchronous, veriloga
`include "constants.vams"
`include "disciplines.vams"

module
VerilogA_SAR_logic_Synchronous(in_comp,clk,d0,d1,d2,d3,d4,d5,d6,d7,regclk,sampl
eclk,vdd,vss);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;
// 8-bits (8-cycles) + 1-cycle for sampling + 1-cycle for Register output = 10-
cycles

inout vdd,vss;
input in_comp, clk;
output d0,d1,d2,d3,d4,d5,d6,d7;
output sampleclk,regclk;

electrical in_comp,clk,d0,d1,d2,d3,d4,d5,d6,d7,regclk,sampleclk,vdd,vss;

real d_0,d_1,d_2,d_3,d_4,d_5,d_6,d_7,sample_en,sar_counter,reg_out;

analog begin

    // Initial State
    @(initial_step) begin
        sample_en = 1;
        d_7 = 0;
        d_6 = 0;
        d_5 = 0;
        d_4 = 0;
        d_3 = 0;
        d_2 = 0;
        d_1 = 0;
        d_0 = 0;
        sar_counter = 9; //10-bit counter
    end

    // Operation: Comparison
    // At Rising Edge of clk, the bits are updated for comparison.
    // At Falling Edge of clk, the Comparator produces the new bit
    (Regeneration)
    @(cross(V(clk) - clk_threshold, +1))
    begin
        if(sar_counter == 9) begin
            sample_en = 1; //1st count is for sampling
            reg_out = 0;
            d_7 = 0; d_6 = 0; d_5 = 0; d_4 = 0; d_3 = 0; d_2 = 0; d_1 = 0;
            d_0 = 0;

```

```

        sar_counter = sar_counter - 1; end

    else if(sar_counter == 8) begin
        d_7 = 1; d_6 = 0; d_5 = 0; d_4 = 0; d_3 = 0; d_2 = 0; d_1 = 0;
        d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 7) begin
        d_7 = V(in_comp); d_6 = 1; d_5 = 0; d_4 = 0; d_3 = 0; d_2 = 0;
        d_1 = 0; d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 6) begin
        d_6 = V(in_comp); d_5 = 1; d_4 = 0; d_3 = 0; d_2 = 0; d_1 = 0;
        d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 5) begin
        d_5 = V(in_comp); d_4 = 1; d_3 = 0; d_2 = 0; d_1 = 0; d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 4) begin
        d_4 = V(in_comp); d_3 = 1; d_2 = 0; d_1 = 0; d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 3) begin
        d_3 = V(in_comp); d_2 = 1; d_1 = 0; d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 2) begin
        d_2 = V(in_comp); d_1 = 1; d_0 = 0;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 1) begin
        d_1 = V(in_comp); d_0 = 1;
        sar_counter = sar_counter - 1; end

    else if(sar_counter == 0) begin
        d_0 = V(in_comp);
        sar_counter = 9; end
end

// Producing the neg edge of the pulse to sample & to get the register output
@(cross(V(clk) - clk_threshold, -1))
begin
    if(sample_en == 1) begin
        sample_en = 0; end

    if(sar_counter == 9) begin
        reg_out = 1; end
end

V(d7) <+ transition(d_7,delay,ttime);
V(d6) <+ transition(d_6,delay,ttime);
V(d5) <+ transition(d_5,delay,ttime);
V(d4) <+ transition(d_4,delay,ttime);
V(d3) <+ transition(d_3,delay,ttime);
V(d2) <+ transition(d_2,delay,ttime);
V(d1) <+ transition(d_1,delay,ttime);
V(d0) <+ transition(d_0,delay,ttime);
V(regclk) <+ transition(reg_out,delay,ttime);
V(sampleclk) <+ transition(sample_en,delay,ttime);

```

```

end
endmodule

```

A.1.4 Output Register (8-bit):

```

// VerilogA for SAR_VerilogA, VerilogA_Register_8bit, veriloga
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_Register_8bit
(clk,in0,in1,in2,in3,in4,in5,in6,in7,out0,out1,out2,out3,out4,out5,out6,out7,
vdd,vss);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;

inout vdd,vss;
input clk,in0,in1,in2,in3,in4,in5,in6,in7;
output out0,out1,out2,out3,out4,out5,out6,out7;

electrical clk,in0,in1,in2,in3,in4,in5,in6,in7,out0,out1,out2,out3,out4,out5,
out6,out7,vdd,vss;
real d_0,d_1,d_2,d_3,d_4,d_5,d_6,d_7;

analog begin
  @(cross(V(clk) - clk_threshold, +1))
  begin
    d_7 = V(in7);
    d_6 = V(in6);
    d_5 = V(in5);
    d_4 = V(in4);
    d_3 = V(in3);
    d_2 = V(in2);
    d_1 = V(in1);
    d_0 = V(in0);
  end

  V(out7) <+ transition(d_7,delay,ttime);
  V(out6) <+ transition(d_6,delay,ttime);
  V(out5) <+ transition(d_5,delay,ttime);
  V(out4) <+ transition(d_4,delay,ttime);
  V(out3) <+ transition(d_3,delay,ttime);
  V(out2) <+ transition(d_2,delay,ttime);
  V(out1) <+ transition(d_1,delay,ttime);
  V(out0) <+ transition(d_0,delay,ttime);
end
endmodule

```

A.1.5 DAC (8-bit):

```

// VerilogA for VerilogA_DAC_8bit
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_DAC_8bit(d0,d1,d2,d3,d4,d5,d6,d7,vout,vdd,vss,vmin,vmax);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;

```

```

inout vdd,vss;
input d0,d1,d2,d3,d4,d5,d6,d7;
input vmin, vmax;
output vout;

electrical vout,vdd,vss,d0,d1,d2,d3,d4,d5,d6,d7,vmin,vmax;
real result,d_0,d_1,d_2,d_3,d_4,d_5,d_6,d_7;

analog begin
    d_7 = V(d7)*128;
    d_6 = V(d6)*64;
    d_5 = V(d5)*32;
    d_4 = V(d4)*16;
    d_3 = V(d3)*8;
    d_2 = V(d2)*4;
    d_1 = V(d1)*2;
    d_0 = V(d0)*1;

    result = ((d_7+d_6+d_5+d_4+d_3+d_2+d_1+d_0) *
              ((V(vmax)-V(vmin))/(256))) + V(vmin) ;

    V(vout) <+ transition(result,delay,ttime);
end
endmodule

```

A.2: Asynchronous SAR ADC:

The sample-and-hold, the clocked comparator, the output register, and the DAC codes mentioned previously are re-used for the asynchronous SAR ADC. The internal clock generator and the SAR logic are designed using VerilogA-based gates.

A.2.1 Internal Clock Generator:

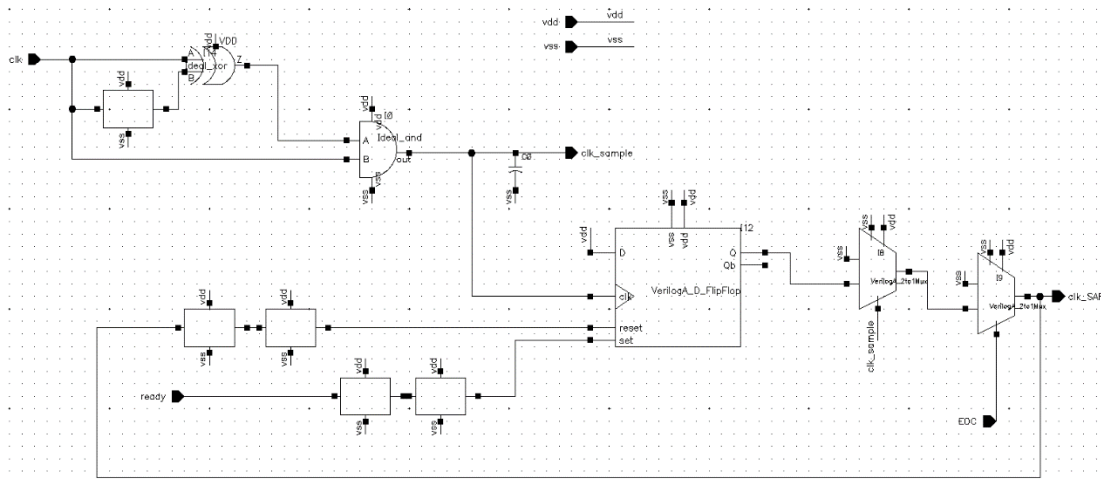


Fig.54. Ideal internal clock generator

A.2.2 SAR Logic:

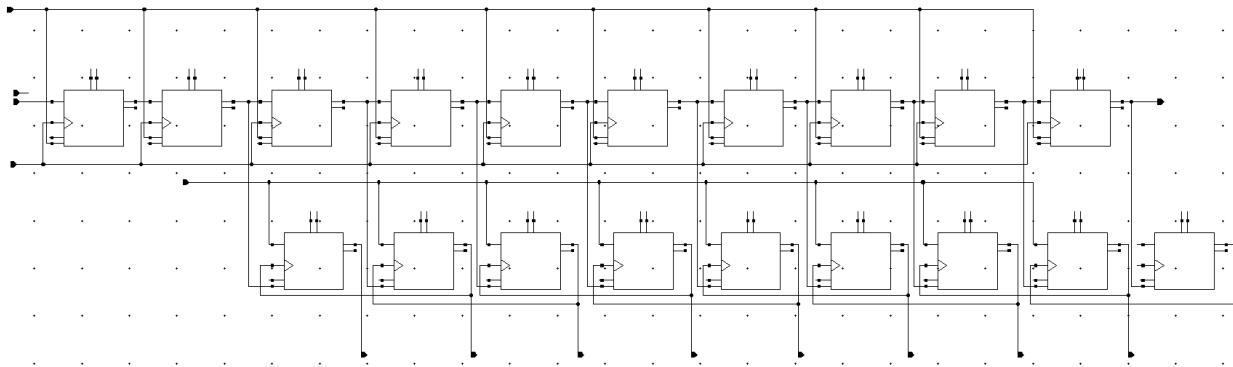


Fig.55. Ideal SAR logic

A.2.3 D-Flipflop:

```
// VerilogA for SAR_VerilogA, VerilogA_D_FlipFlop, verilogA
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_DFlipFlop(D,clk,Q,Qb,set,reset,vdd,vss);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;

inout vdd,vss;
input D,clk,set,reset;
output Q,Qb;

electrical D,clk,Q,Qb,set,reset,vdd,vss;
real d_0,d_0b;

analog begin
    @(cross(V(set) - vtrans, +1)) begin
        d_0 = 1;
        d_0b = 0; end
    @(cross(V(reset) - vtrans, +1)) begin
        d_0 = 0;
        d_0b = 1; end

    @(cross(V(clk) - clk_threshold, +1))
    begin
        //if((V(set) < vtrans) && (V(reset) < vtrans) ) begin
        if(V(D) > vtrans) begin
            d_0 = 1; d_0b = 0; end

            else begin
                d_0 = 0; d_0b = 1; end
            //end
        end

        V(Q) <+ transition(d_0,delay,ttime);
        V(Qb) <+ transition(d_0b,delay,ttime);
    end
endmodule
```

A.2.4 Inverter Gate:

```
// VerilogA for SAR_VerilogA_Asynchronous, VerilogA_inv, veriloga
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_inv(A,X);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;

input A;
output X;

electrical A,X;
real X_out;

analog begin

    if(V(A) > vtrans) begin
        X_out = 0; end
    else begin
        X_out = 1; end

    V(X) <+ transition(X_out,delay,ttime);
end
endmodule
```

A.2.5 XOR Gate:

```
// VerilogA for SAR_VerilogA, VerilogA_XOR, veriloga
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_XOR(A,B,X);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;

input A,B;
output X;

electrical A,B,X;
real X_out;

analog begin

    if(V(A) == V(B)) begin
        X_out = 0; end

    else begin
        X_out = 1; end

    V(X) <+ transition(X_out,delay,ttime);
end
endmodule
```

A.2.6 AND Gate:

```
// VerilogA for SAR_VerilogA, VerilogA_AND, veriloga
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_AND(A,B,X);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;

input A,B;
output X;

electrical A,B,X;
real X_out;

analog begin

    if(V(A) < vtrans) begin
        X_out = 0; end
    else if(V(B) < vtrans) begin
        X_out = 0; end
    else begin
        X_out = 1; end

    V(X) <+ transition(X_out,delay,ttime);
end
endmodule
```

A.2.7 Multiplexer (2-to-1):

```
// VerilogA for SAR_VerilogA_Asynchrous, VerilogA_2to1Mux, veriloga
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_2to1Mux(in0,in1,select,out,vdd,vss);

parameter real vtrans=0.5;
parameter real delay = 0;
parameter real ttime = 1p;
parameter real clk_threshold = 0.5;

inout vdd,vss;
input in0,in1;
output out;

electrical in0,in1,select,out,vdd,vss;
real result;

analog begin

    if(V(select) < vtrans) begin
        result = V(in0); end
    else begin
        result = V(in1); end

    V(out) <+ transition(result,delay,ttime);

end
endmodule
```

A.2.8 Delay Cell (1ns):

```
// VerilogA for SAR_VerilogA_Asynchrous, VerilogA_1ns_delaycell, veriloga
`include "constants.vams"
`include "disciplines.vams"

module VerilogA_1ns_delaycell(inp,outp,vdd,vss);

parameter real vtrans=0.5;
parameter real delay = 1n;
parameter real ttime = 1f;

inout vdd,vss;
input inp;
output outp;

electrical inp,outp,vdd,vss;
real d_out;

analog begin
    @(cross(V(inp) - vtrans, +1))
    begin
        d_out = 1;
    end

    @(cross(V(inp) - vtrans, -1))
    begin
        d_out = 0;
    end

    V(outp) <+ transition(d_out,delay,ttime);
end

endmodule
```


References

- [1] J. Fredenburg and M. P. Flynn, "ADC trends and impact on SAR ADC architecture and analysis," 2015 IEEE Custom Integrated Circuits Conference (CICC), San Jose, CA, 2015, pp. 1-8.
- [2] S. Zhang, K. Huang and H. Shen, "A Robust 8-Bit Non-Volatile Computing-in-Memory Core for Low-Power Parallel MAC Operations," IEEE Transactions on Circuits and Systems I: Regular Papers, 2020.
- [3] W. Mao, Y. Li, C. Heng, and Y. Lian, "A Low Power 12-bit 1-kS/s SAR ADC for Biomedical Signal Processing," IEEE Transactions on Circuits and Systems I: Regular Papers, vol. 66, no. 2, pp. 477-488, Feb. 2019.
- [4] P. J. A. Harpe et al., "A 26 μ W 8 bit 10 MS/s Asynchronous SAR ADC for Low Energy Radios," IEEE Journal of Solid-State Circuits, vol. 46, no. 7, pp. 1585-1595, July 2011.
- [5] C. Huang, J. Lin, Y. Shyu and S. Chang, "A Systematic Design Methodology of Asynchronous SAR ADCs," IEEE Transactions on Very Large Scale Integration (VLSI) Systems, vol. 24, no. 5, pp. 1835-1848, May 2016, doi: 10.1109/TVLSI.2015.2494063.
- [6] O. Kardonik, "A study of SAR ADC and implementation of 10-bit asynchronous design," M.S. Thesis, Department of Electrical and Computer Engineering, University of Texas at Austin, 2013.
- [7] B. Razavi, "Data Conversion System Design," IEEE Press, 1995.
- [8] M. Dessouky and A. Kaiser, "Very low-voltage digital-audio $\Delta\Sigma$ modulator with 88-dB dynamic range using local switch bootstrapping," IEEE Journal of Solid-State Circuits, vol. 36, no. 3, pp. 349-355, March 2001.
- [9] C. Liu, S. Chang, G. Huang, and Y. Lin, "A 10-bit 50-MS/s SAR ADC with a Monotonic Capacitor Switching Procedure," in IEEE Journal of Solid-State Circuits, vol. 45, no. 4, pp. 731-740, April 2010.
- [10] B. Razavi, "The StrongARM Latch [A Circuit for All Seasons]," IEEE Solid-State Circuits Magazine, vol. 7, pp. 12-17, 2015.
- [11] M. Aldacher, M. Nasrollahpour and S. Hamed-hagh, "A low-power, high-resolution, 1 GHz differential comparator with low-offset and low-kickback," 2017 24th IEEE International Conference on Electronics, Circuits and Systems (ICECS), Batumi, 2017, pp. 310-313.
- [12] M. Al-Qadasi, A. Alshehri, A. S. Almansouri, T. Al-Attar, and H. Fariborzi, "A High Speed Dynamic StrongARM Latch Comparator," 2018 IEEE 61st International Midwest Symposium on Circuits and Systems (MWSCAS), Windsor, ON, Canada, 2018, pp. 540-541.
- [13] M. M. Ayesh, S. Ibrahim, and M. M. Aboudina, "Design and analysis of a low-power high-speed charge-steering based StrongARM comparator," 2016 28th International Conference on Microelectronics (ICM), Giza, Egypt, 2016, pp.209-212.

- [14] T.O. Anderson. Optimum control logic for successive approximation analog-to-digital converters. Communications Systems Research Section, JPL Technical Report 32-1526, Vol XIII.
- [15] J. Yuan and C. Svensson, "New single-clock CMOS latches and flipflops with improved speed and power savings," in *IEEE Journal of Solid-State Circuits*, vol. 32, no. 1, pp. 62-69, Jan. 1997.
- [16] W. Kester, "Understand SINAD, ENOB, SNR, THD, THD + N, and SFDR so you don't get lost in the noise floor," Analog Devices, Norwood, MA, USA, Tech. Rep. MT-003, 2009.